大数据，成就未来



使用scikit-learn构建模型

2026/10/22

雨课堂 Rain Classroom

使用scikit-learn构建模型

本章学习目标

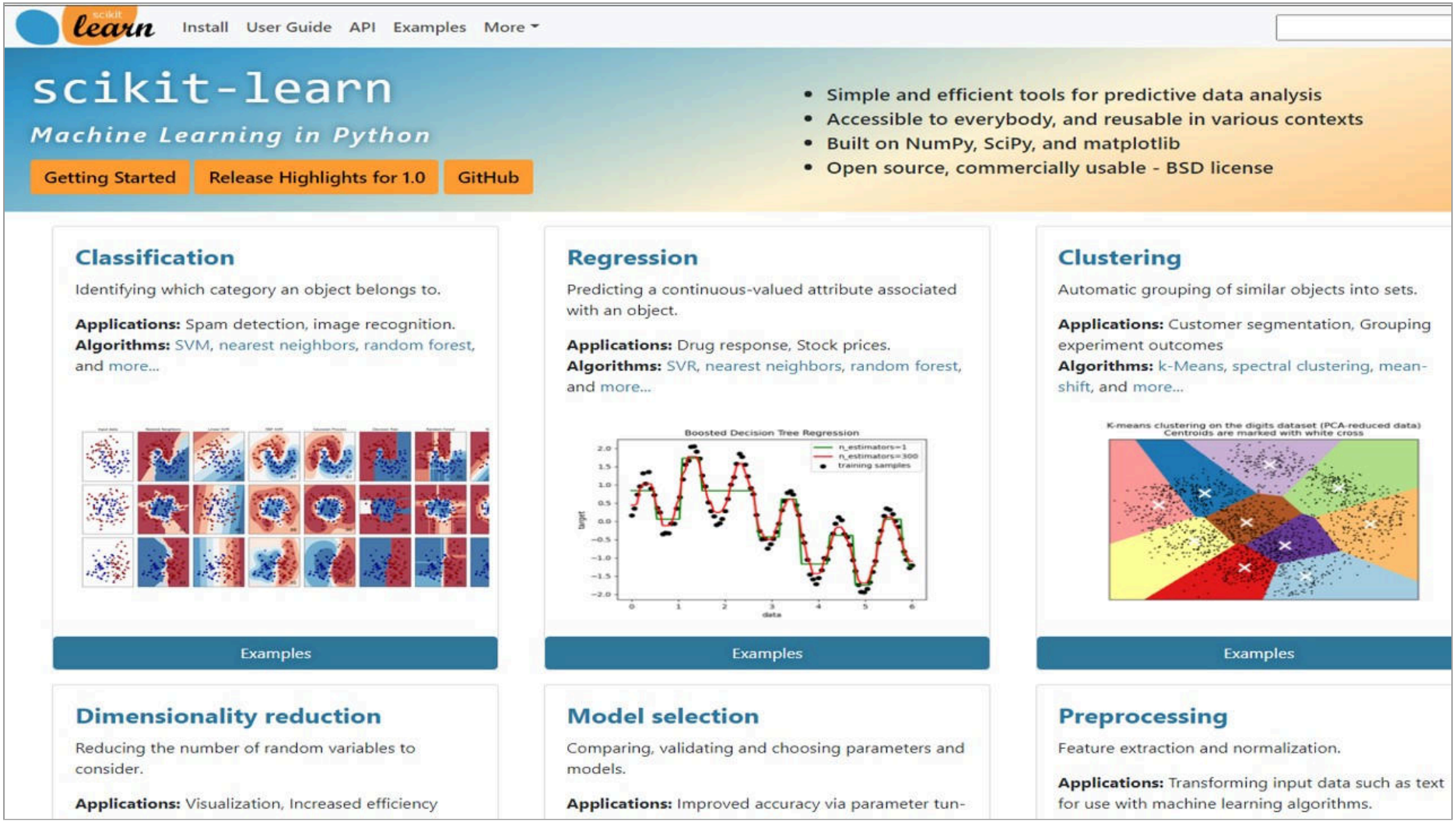
（1）掌握sklearn转换器、估计器的使用

（2）掌握sklearn数据标准化与数据划分

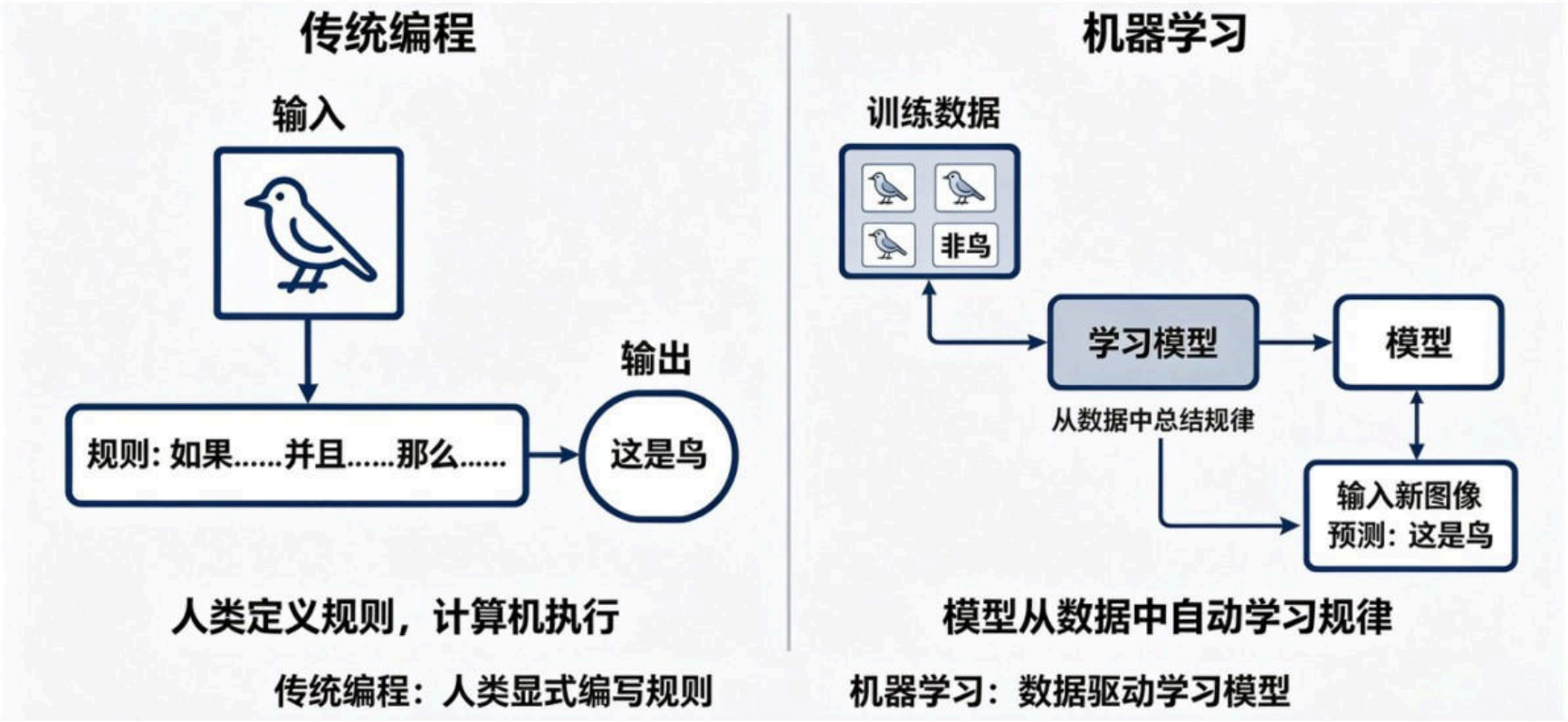
（3）掌握sklearn中聚类、分类、回归模型的构建

（4）掌握sklearn中聚类、分类、回归模型的评价。

- scikit-learn(简称sklearn)库整合了多种机器学习的算法，可以帮助使用者在数据分析过程中快速建立模型，且模型接口统一，使用起来非常方便。
- 同时，sklearn拥有优秀的官方文档（**http://scikit-learn.org/stable/**），知识点详尽，内容丰富。


scikit learn
Install
User Guide
API
Examples
More
scikit-learn
Machine Learning in Python
Getting Started
Release Highlights for 1.0
GitHub
Simple and efficient tools for predictive data analysis
Accessible to everybody, and reusable in various contexts
Built on NumPy, SciPy, and matplotlib
Open source, commercially usable - BSD license
Classification
Identifying which category an object belongs to.
Applications: Spam detection, image recognition.
Algorithms: SVM, nearest neighbors, random forest, and more...
Examples
Regression
Predicting a continuous-valued attribute associated with an object.
Applications: Drug response, Stock prices.
Algorithms: SVR, nearest neighbors, random forest, and more...
Boosted Decision Tree Regression
n_estimators=1
n_estimators=300
training samples
target
data
Examples
Clustering
Automatic grouping of similar objects into sets.
Applications: Customer segmentation, Grouping experiment outcomes
Algorithms: k-Means, spectral clustering, mean-shift, and more...
K-means clustering on the digits dataset (PCA-reduced data)
Centroids are marked with white cross
Examples
Dimensionality reduction
Reducing the number of random variables to consider.
Applications: Visualization, Increased efficiency
Model selection
Comparing, validating and choosing parameters and models.
Applications: Improved accuracy via parameter tun-
Preprocessing
Feature extraction and normalization.
Applications: Transforming input data such as text for use with machine learning algorithms.

范式转变：从“规则”到“模式”—— 两种编程，两种思维



PART 3 核心突破——走进机器学习与深度学习

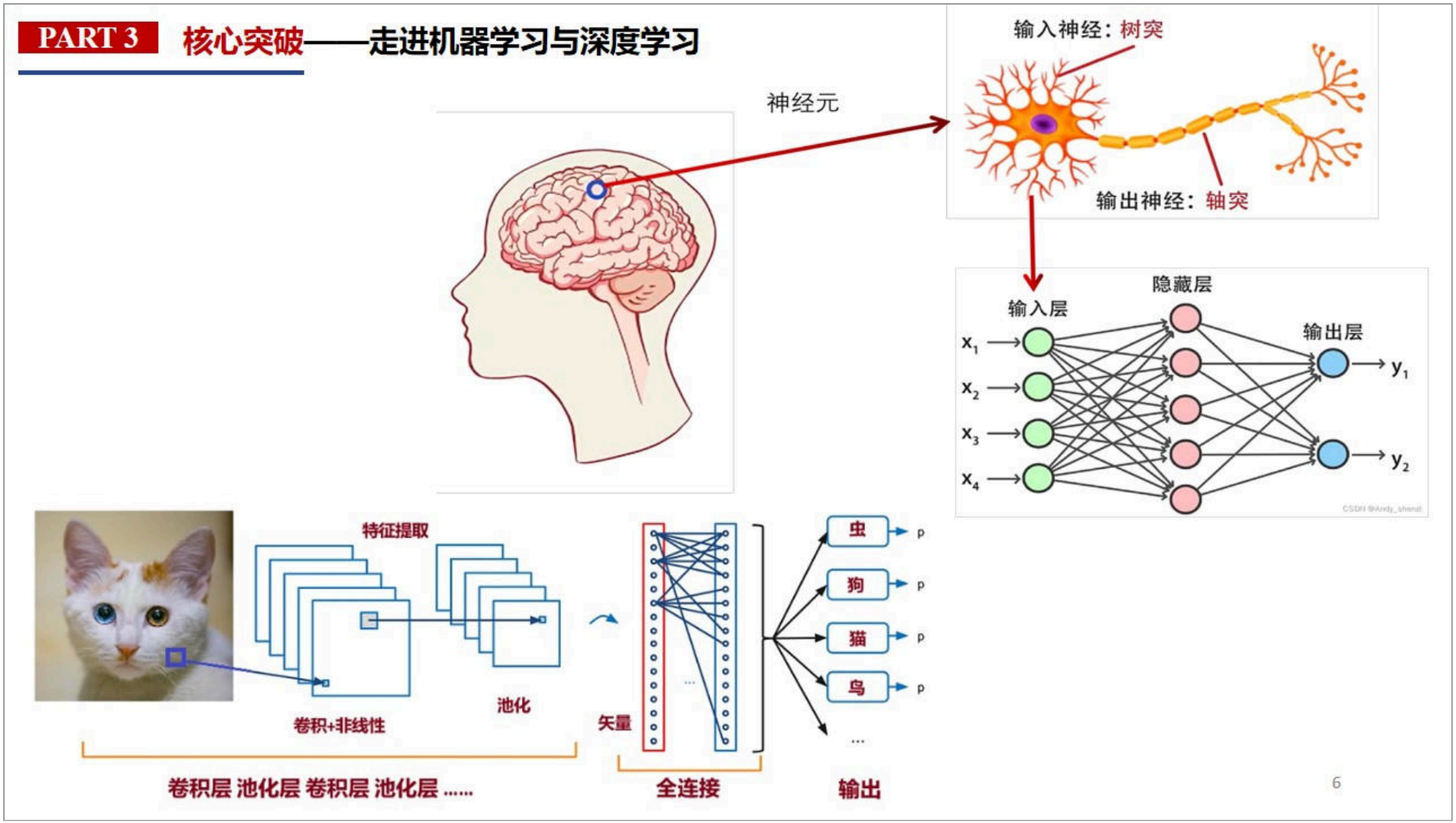
机器学习的流派：监督学习、无监督学习 & 强化学习



监督学习






PART 3
核心突破——走进机器学习与深度学习
神经元
输入神经：树突
输出神经：轴突
隐藏层
输入层
输出层
特征提取
卷积+非线性
池化
矢量
虫
狗
猫
鸟
卷积层 池化层 卷积层 池化层 .....
全连接
输出
6

sklearn简介（补充）

机器学习按照学习方式可以分为几大类:

① **监督学习**：数据带有一个**附加属性(标记)**，即我们想要预测的结果值，这类问题可以是:

- **分类问题**：样本属于两个或更多个类，从已经标记的数据中学习如何预测未标记数据的类别。
  - 例如，手写数字识别，将每个输入向量分配给有限数目的离散类别之一。
  - 分类问题是监督学习的一个离散形式（区别于连续形式），从有限的类别中，给每个样本贴上正确的标签。
- **回归问题**：如果期望的输出由一个或多个连续变量组成，则该任务称为 **回归** 。

➢ **监督学习常用算法**：线性回归、逻辑回归、随机森林、决策树、梯度下降决策树、支持向量机(SVM)、神经网络、临近邻居等

sklearn简介（补充）

机器学习按照学习方式可以分为几大类:

② **无监督学习**：训练数据由没有任何相应目标值的一组输入向量x组成。

- 这种问题的目标可能是在数据中发现彼此类似的示例所聚成的组，这种问题称为 **聚类**；
- 或者，确定输入空间内的数据分布，称为 **密度估计** ；
- 又或从高维数据投影数据空间 **（降维）** 缩小到二维或三维以进行可视化。

➢ **无监督学习常用算法:**

- K-Means聚类、关联规则(Association Rule)
- PCA(主成分分析)、t-SNE(分布式随机邻域嵌入)、

8

sklearn简介（补充）

机器学习按照学习方式可以分为几大类:

③ **半监督学习:**

- 使用一组数量较小的有标记的样本数据来完成某些操作的要求（如自我训练）
- 使用标记样本数量训练出来的模型为部分训练模型（Partial-trained Model），该模型的任务是获取未标记数据的标记，其结果被认为是伪标记数据。
- 最后结合标记和伪标记的数据集，再进行训练。这创建了一种独特的算法，该算法结合了监督学习和无监督学习的描述性和预测性。

➢ **半监督学习常用模型:**

- 图论推理算法（Graph Inference）、拉普拉斯支持向量机（Laplacian SVM.）、概率生成模型等。

9

雨课堂 Rain Classroom

sklearn简介（补充）

机器学习按照学习方式可以分为几大类:

③ **强化机器学习:**

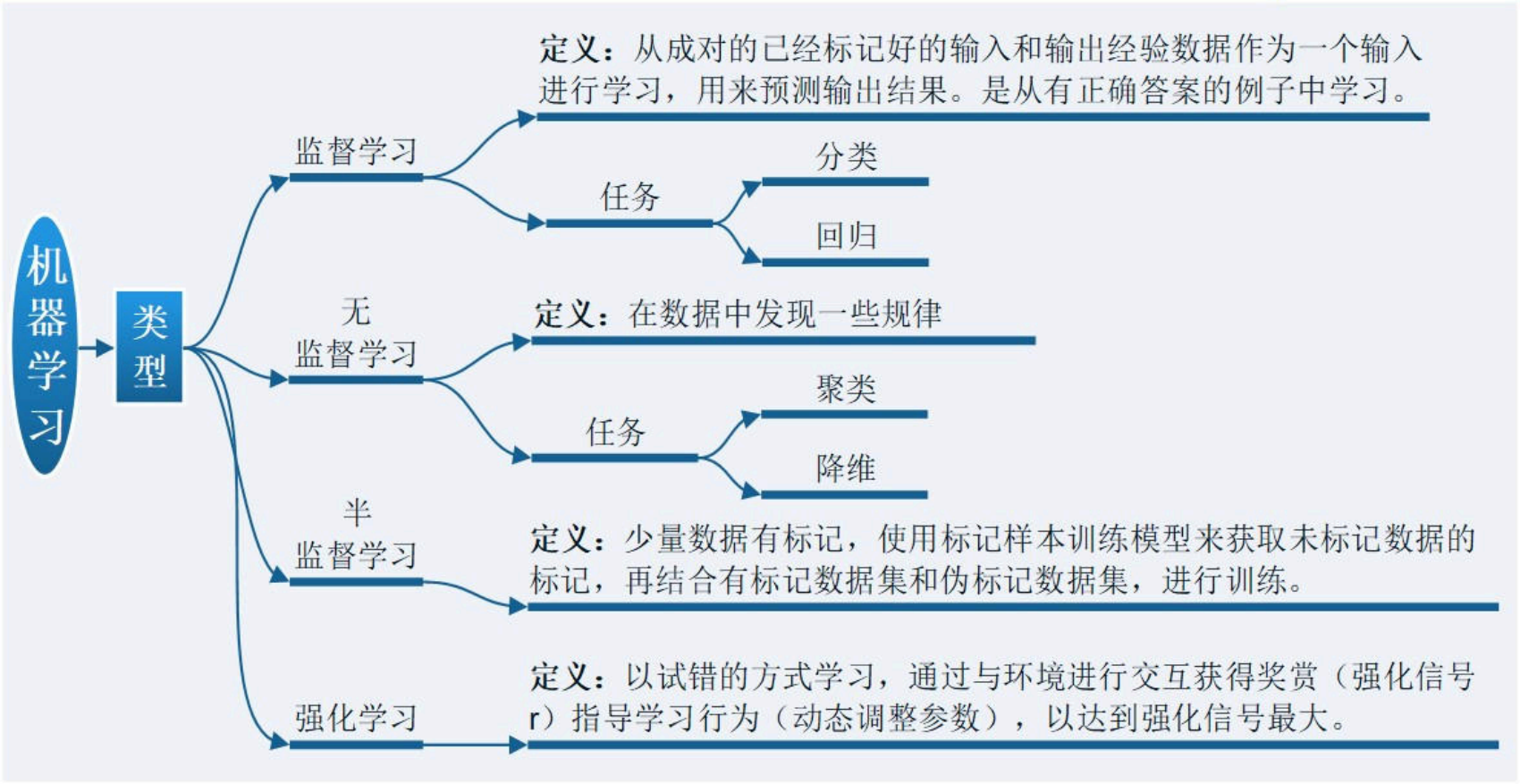
- 智能体（Agent）以“试错”的方式进行学习，通过与环境进行交互获得的奖赏（强化信号r）指导行为，目标是使智能体获得最大的奖赏；
- 强化学习系统学习的目标是动态地调整参数，以达到强化信号最大。若已知r/A梯度信息，则可直接使用监督学习算法。但因为强化信号r与Agent产生的动作A没有明确的函数形式描述，所以梯度信息r/A无法得到。因此，在强化学习系统中，需要某种随机单元，使用这种随机单元，Agent在可能动作空间中进行搜索并发现正确的动作。

➢ **强化学习常用模型:**

- 强化学习的常见模型是标准的马尔可夫决策过程；
- Q-Learning、TD（瞬时差分）、MCTS（蒙特卡洛树搜索）、A3C等

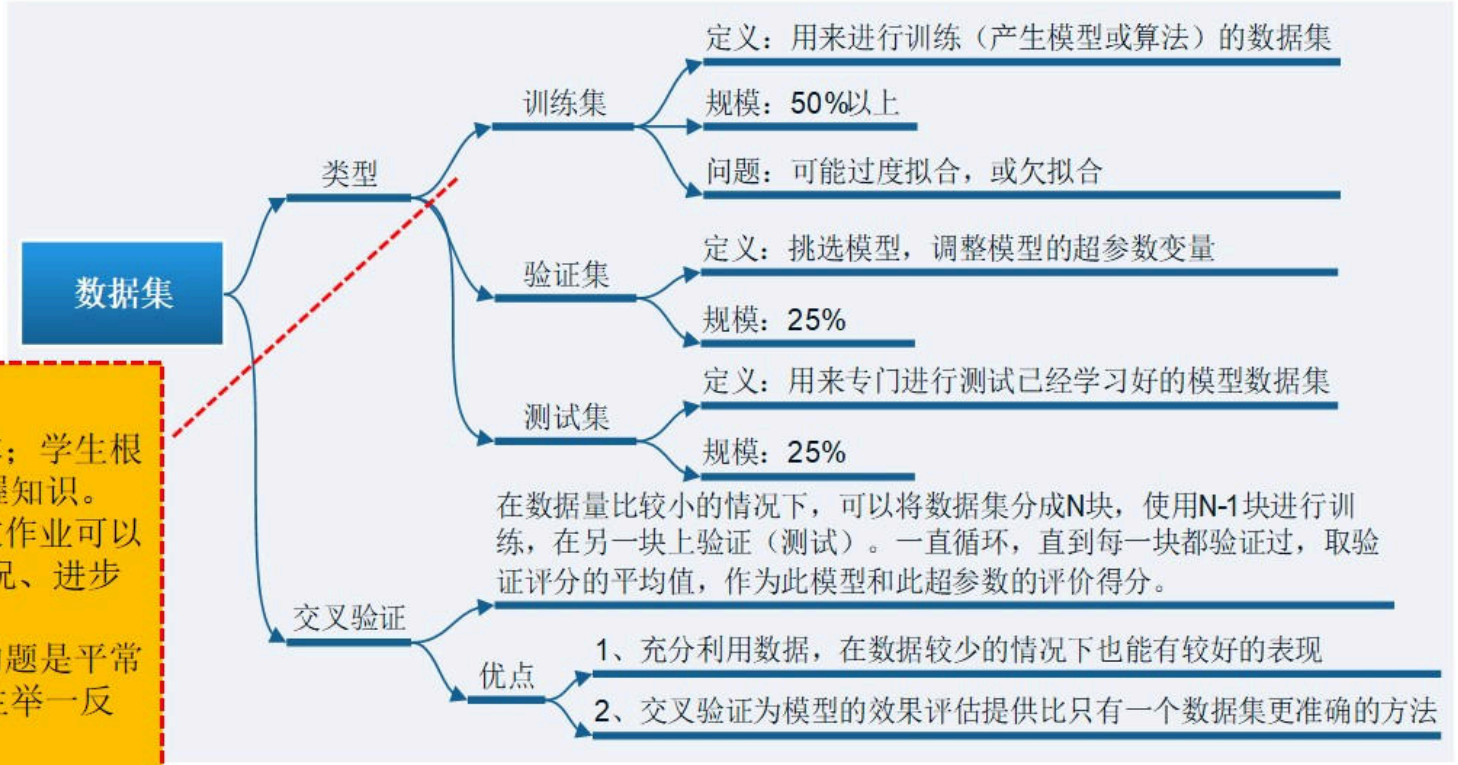
10

sklearn简介（补充）



sklearn简介（补充）

机器学习的认识



一个形象的比喻：

训练集---学生的课本；学生根据课本里的内容来掌握知识。

验证集---作业，通过作业可以知道 不同学生学习情况、进步的速度快慢。

测试集---考试，考的题是平常都没有见过，考察学生举一反三的能力。

sklearn简介（补充）

机器学习的认识



13

sklearn简介（补充）

sklearn库官方文档结构

- Tutorials：是一个官方教程，可以理解快速上手教程
- User guide（用户指南）：对每一个算法的详细介绍
- API：库调用的方法
- FAQ：常见问题
- contributing：贡献，还介绍最新的一些代码、功能

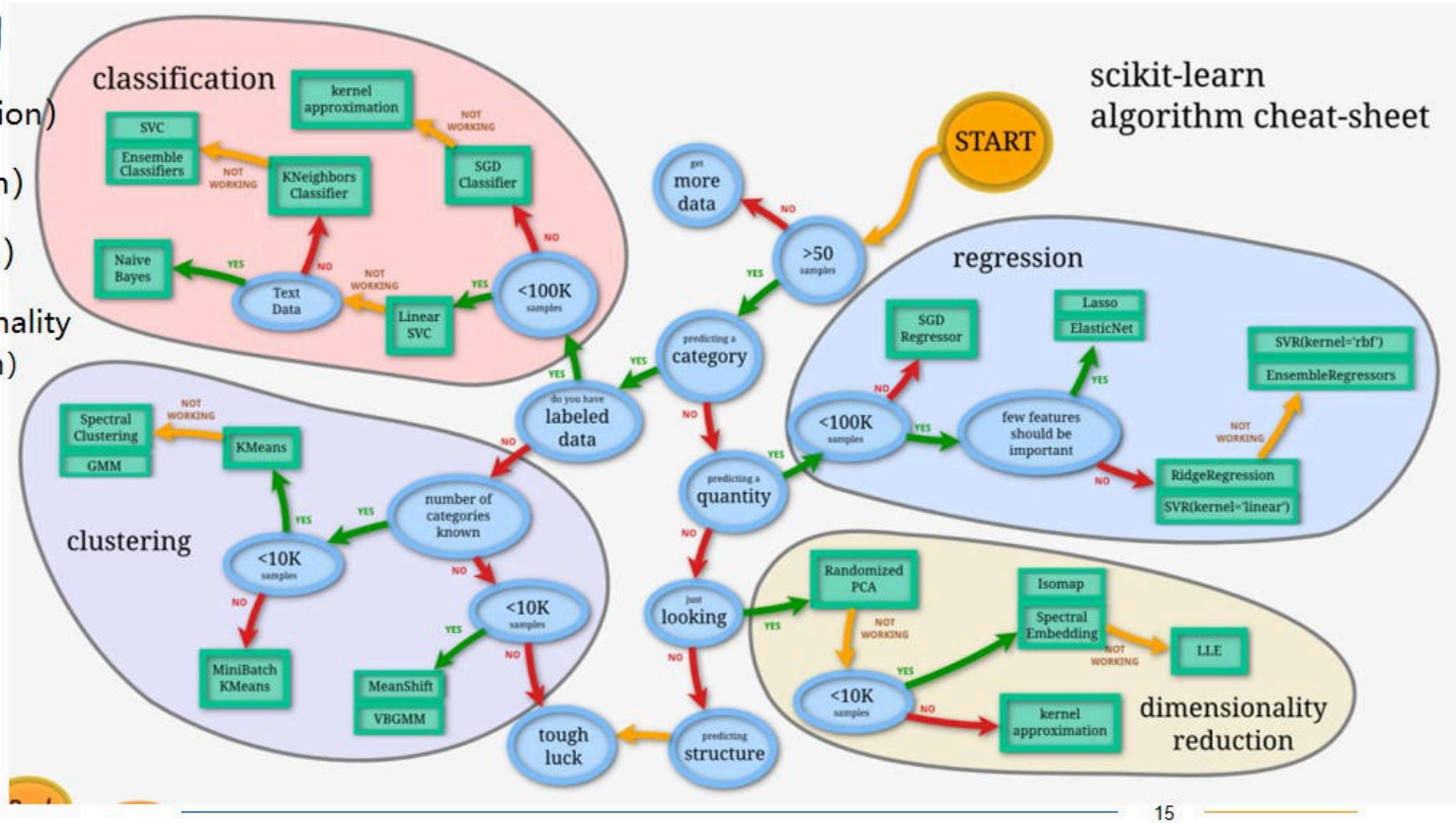


- 总结：一般的做法是API里面找到你要调用的方法，然后可以查看方法参数的情况和使用情况。也可以在指南里面找到具体的解释。

sklearn简介（补充）

sklearn库结构

- 分类（classification）
- 回归（regression）
- 聚类（clustering）
- 降维（dimensionality reduction）



雨课堂 Rain Classroom

sklearn简介（补充）

机器学习主要步骤中sklearn库应用

1. **数据集**：面对自己的任务肯定有自己的数据集，但是对于初学者来说，sklearn提供了一些数据
   - 小规模数据集：数据包含在 datasets 里（datasets.load_*()）
   - 大规模数据集：需要从网络上下载（datasets.fetch_*()）
   - 本地生成数据集（datasets.make_*()）
2. **数据预处理**：数据预处理包括：降维、数据归一化、特征提取和特征转换（one-hot）等，在sklearn里有很多方法，具体查看API。
3. **选择模型并训练**： sklearn里面有很多的机器学习方法，可以查看API找到你需要的方法，sklearn统一了所有模型调用的API，使用起来比较简单。
4. **模型评分**
5. **模型的保存与恢复**

16

雨课堂 Rain Classroom

目录


1
使用sklearn转换器处理数据
2
构建并评价聚类模型
3
构建并评价分类模型
4
构建并评价回归模型
5
小结

17

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- sklearn库的datasets模块集成了部分数据分析的经典数据集，可以使用这些数据集进行数据预处理，建模等操作，熟悉sklearn的数据处理流程和建模流程。
- sklearn.datasets模块主要提供了一些**导入（load）**、**在线下载（fetch）**及**本地生成（make）**数据集的方法，可以通过dir或help命令查看，我们会发现主要有三种形式：
  - **load**_<dataset_name>
  - **fetch**_<dataset_name>
  - **make**_<dataset_name>

雨课堂 Rain Classroom

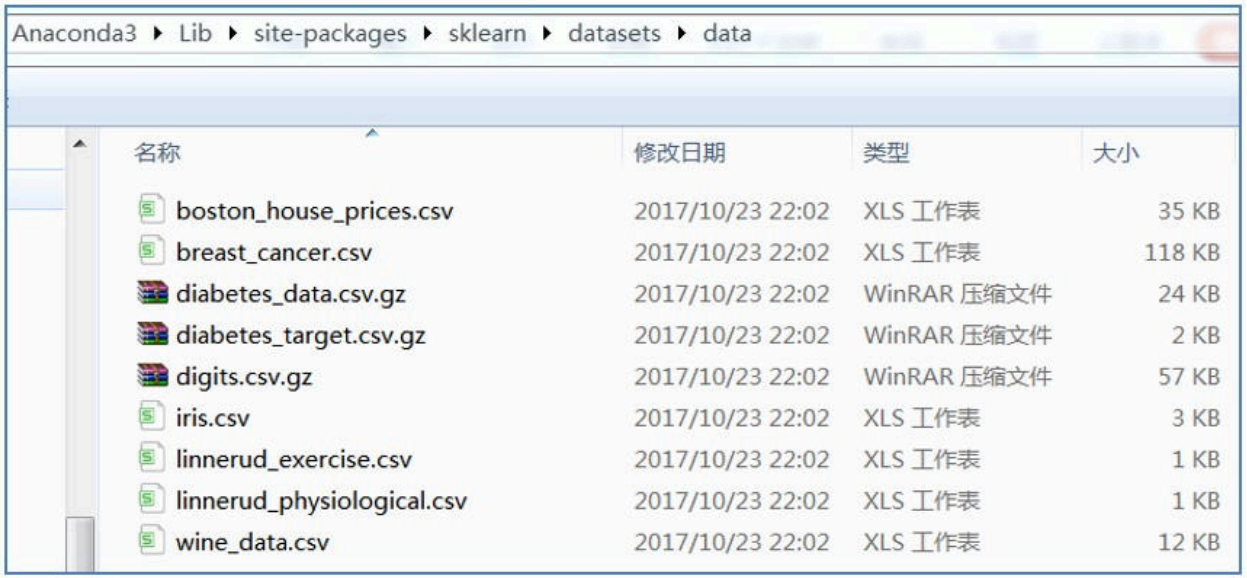
加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- load_<dataset_name>

```
In [1]: from sklearn import datasets

In [2]: datasets.load_*?
datasets.load_boston
datasets.load_breast_cancer
datasets.load_diabetes
datasets.load_digits
datasets.load_files
datasets.load_iris
datasets.load_linnerud
datasets.load_mlcomp
datasets.load_sample_image
datasets.load_sample_images
datasets.load_svmlight_file
datasets.load_svmlight_files
datasets.load_wine
```



加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- load_<dataset_name>
  - datasets.load_boston ：波士顿房价数据集（回归）
  - datasets.load_breast_cancer ：乳腺癌数据集（分类）
  - datasets.load_diabetes ：糖尿病数据集（回归）
  - datasets.load_digits ：手写数字数据集（分类）
  - datasets.load_files ：加载带有类别的文本文件作为子文件夹名称
  - datasets.load_iris ：鸢尾花数据集（分类）
  - datasets.load_linnerud ：体能训练数据集

雨课堂 Rain Classroom

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- load_<dataset_name>
  - datasets.load_mlcomp ：被弃用了
  - datasets.load_sample_image ：加载单个样本图像的numpy数组
  - datasets.load_sample_images ：加载样本图像以进行图像处理。
  - datasets.load_svmlight_file ：将svmlight / libsvm格式的数据集加载到稀疏CSR矩阵中
  - datasets.load_svmlight_files ：以SVMlight格式从多个文件加载数据集
  - datasets.load_wine ：葡萄酒数据集（分类）

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- fetch_<dataset_name>

```
In [3]: datasets.fetch_*?
datasets.fetch_20newsgroups
datasets.fetch_20newsgroups_vectorized
datasets.fetch_california_housing
datasets.fetch_covtype
datasets.fetch_kddcup99
datasets.fetch_lfw_pairs
datasets.fetch_lfw_people
datasets.fetch_mldata
datasets.fetch_olivetti_faces
datasets.fetch_rcv1
datasets.fetch_species_distributions
```

datasets.fetch_20newsgroups_vectorized([...])	Load the 20 newsgroups da (classification).
datasets.fetch_california_housing([...])	Load the California housing
datasets.fetch_covtype([data_home, ...])	Load the covertype dataset
datasets.fetch_kddcup99([subset, data_home, ...])	Load the kddcup99 dataset
datasets.fetch_lfw_pairs([subset, ...])	Load the Labeled Faces in t (classification).
datasets.fetch_lfw_people([data_home, ...])	Load the Labeled Faces in t (classification).
datasets.fetch_olivetti_faces([data_home, ...])	Load the Olivetti faces data-
datasets.fetch_openml([name, version, ...])	Fetch dataset from openml
datasets.fetch_rcv1([data_home, subset, ...])	Load the RCV1 multilabel da
datasets.fetch_species_distributions([...])	Loader for species distributi

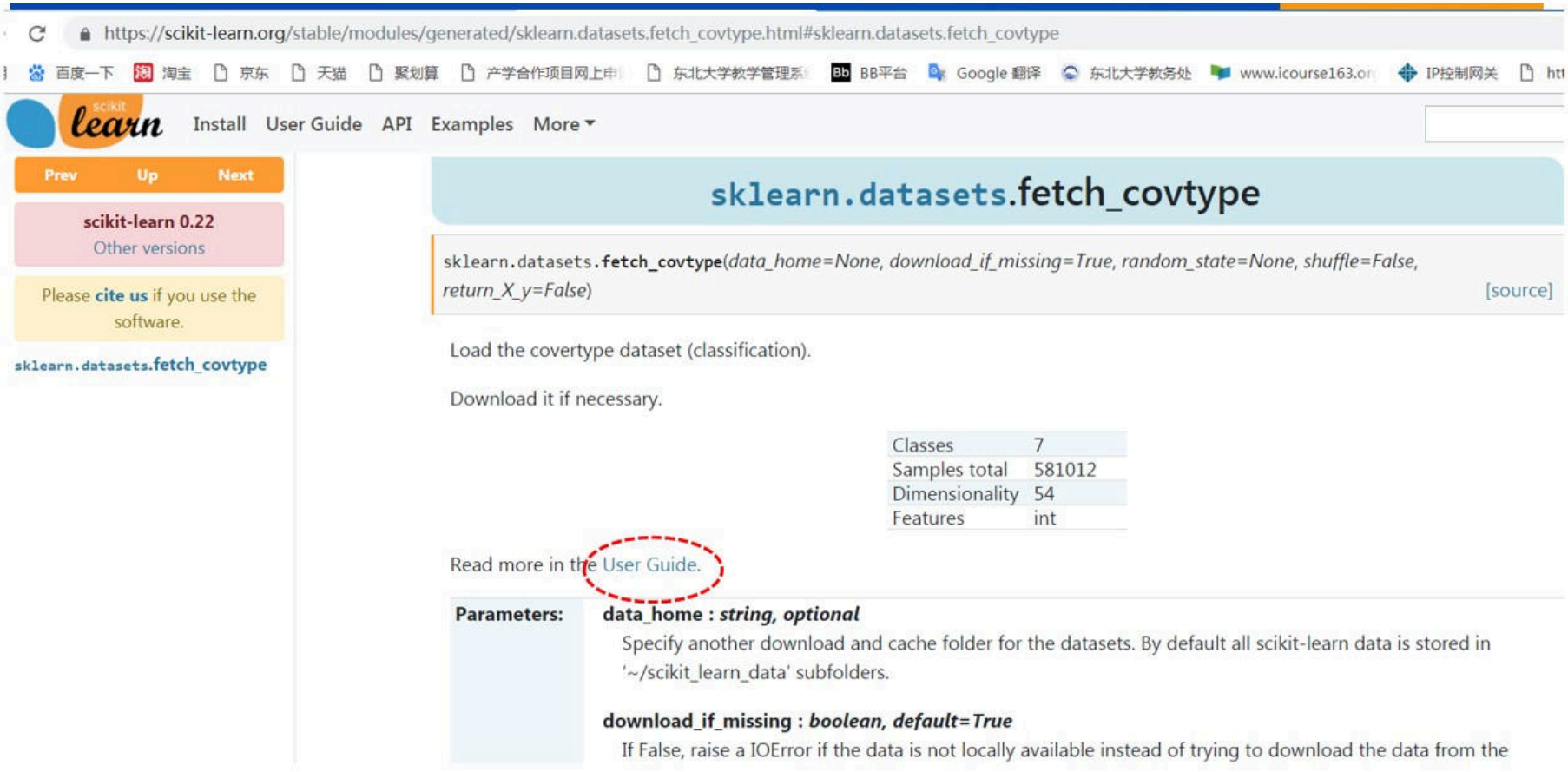
22

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- fetch_<dataset_name>
  - datasets.fetch_20newsgroups : 20个新闻组数据集（分类）
  - datasets.fetch_20newsgroups_vectorized : 20个新闻组数据集并将其矢量化为令牌计数（分类）
  - datasets.fetch_california_housing : 加州住房数据集（回归）
  - datasets.fetch_covtype : 隐色数据集（分类）
  - datasets.fetch_kddcup99 : kddcup99数据集（分类）
  - datasets.fetch_lfw_pairs : 加载标签面在野外（LFW）对数据集（分类）
  - datasets.fetch_lfw_people : 带标签的人脸数据集（分类）
  - datasets.fetch_mldata : machine learning data set repository
  - datasets.fetch_olivetti_faces : Olivetti脸部图像数据集（分类）
  - datasets.fetch_rcv1 : 路透社新闻语料数据集
  - datasets.fetch_species_distributions : 物种分布数据集

加载datasets模块中数据集



加载datasets模块中数据集

5.3.4. Forest covertypes

The samples in this dataset correspond to 30×30m patches of forest in the US, collected for the task of predicting each patch's cover type, i.e. the dominant species of tree. There are seven covertypes, making this a multiclass classification problem. Each sample has 54 features, described on the dataset's homepage. Some of the features are boolean indicators, while others are discrete or continuous measurements.

Data Set Characteristics:

Classes	7
Samples total	581012
Dimensionality	54
Features	int

`sklearn.datasets.fetch_covtype` will load the covertype dataset; it returns a dictionary-like object with the feature matrix in the `data` member and the target values in `target`. The dataset will be downloaded from the web if necessary.

25

加载datasets模块中数据集

UCI Machine Learning Repository
Center for Machine Learning and Intelligent Systems

Covertype Data Set

Download: Data Folder, Data Set Description

Abstract: Forest CoverType dataset



Data Set Characteristics:	Multivariate	**Number of Instances:**	581012	**Area:**	Life
Attribute Characteristics:	Categorical, Integer	**Number of Attributes:**	54	**Date Donated**	1998-08-01
Associated Tasks:	Classification	**Missing Values?**	No	**Number of Web Hits:**	199164

Source:

Original Owners of Database:

Remote Sensing and GIS Program
Department of Forest Sciences
College of Natural Resources
Colorado State University
Fort Collins, CO 80523
(contact Jock A. Blackard, jblackard '@' fs.fed.us or Dr. Denis J. Dean, denis.dean '@' utdallas.edu)

26

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- **make**_<dataset_name>
- scikit-learn包括各种随机样本生成器，可以用来建立可控制大小和复杂性的人工数据集，包括：
- **分类和聚类生成器：**产生一个相应特征的离散矩阵
  - 单标签：如make_blobs/make_classification通过分配每个类的一个或多个正态分布的点的群集创建的多类数据集。
  - 多标签：如make_multilabel_classification 生成多个标签的随机样本，反映从一个混合主题中引用一个词袋。
- **回归生成器**
- **流形学习生成器**

[python]

```
In [4]: datasets.make_*?
datasets.make_biclusters
datasets.make_blobs
datasets.make_checkerboard
datasets.make_circles
datasets.make_classification
datasets.make_friedman1
datasets.make_friedman2
datasets.make_friedman3
datasets.make_gaussian_quantiles
datasets.make_hastie_10_2
datasets.make_low_rank_matrix
datasets.make_moons
datasets.make_multilabel_classification
datasets.make_regression
datasets.make_s_curve
datasets.make_sparse_coded_signal
datasets.make_sparse_spd_matrix
datasets.make_sparse_uncorrelated
datasets.make_spd_matrix
datasets.make_swiss_roll
```


雨课堂 Rain Classroom

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- datasets模块常用数据集的加载函数与解释如下表所示。

数据集加载函数	数据集任务类型	数据集加载函数	数据集任务类型
load_ boston	回归	load_breast_cancer	分类，聚类
fetch_california_housing	回归	load_iris	分类，聚类
load_digits	分类	load_wine	分类

```
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()                    #将数据集赋值给cancer变量
print('breast_cancer数据集的长度为：',len(cancer))
print('breast_cancer数据集的类型为：',type(cancer))
```

```
breast_cancer数据集的长度为：  7
breast_cancer数据集的类型为：  <class 'sklearn.utils.Bunch'>
```

28

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- 加载后的数据集可以视为一个字典，几乎所有的sklearn数据集均可以使用 **data**，**target**，**feature_names**，**DESCR**分别获取数据集的数据，标签，特征名称和描述信息。

```
cancer_data = cancer['data']
print('breast_cancer数据集的数据为：\n', cancer_data)
```

```
breast_cancer数据集的数据为：
 [[1.799e+01 1.038e+01 1.228e+02 ... 2.654e-01 4.601e-01 1.189e-01]
 [2.057e+01 1.777e+01 1.329e+02 ... 1.860e-01 2.750e-01 8.902e-02]
 [1.969e+01 2.125e+01 1.300e+02 ... 2.430e-01 3.613e-01 8.758e-02]
 ...
 [1.660e+01 2.808e+01 1.083e+02 ... 1.418e-01 2.218e-01 7.820e-02]
 [2.060e+01 2.933e+01 1.401e+02 ... 2.650e-01 4.087e-01 1.240e-01]
 [7.760e+00 2.454e+01 4.792e+01 ... 0.000e+00 2.871e-01 7.039e-02]]
```

29

雨课堂 Rain Classroom

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- 加载后的数据集可以视为一个字典，几乎所有的sklearn数据集均可以使用data，**target**，feature_names，DESCR分别获取数据集的数据，标签，特征名称和描述信息。

```
cancer_target = cancer['target']          # 取出数据集的标签
print('breast_cancer数据集的标签为：\n',cancer_target)
```

```
breast_cancer数据集的标签为：
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0
 1 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1 1 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 1 1
 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 1 1 1 0 1
 1 1 1 1 1 1 1 1 0 1 1 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 0 1 1 0 0 0 1 0
 1 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 1 0 1 0 1 1 0 1 0 0 0 0 1 1 0 0 1 1
 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 1 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1
 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 0 1 1
 1 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0 0
 0 1 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 0 0 1 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 1 0 1 1
 0 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1
 1 1 1 1 1 1 0 1 0 1 1 0 1 1 1 1 1 1 0 0 1 0 1 0 1 1 1 1 1 0 1 1 0 1 0 1 0 0
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 0 0 0 0 0 0 1]
```


雨课堂 Rain Classroom

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- 加载后的数据集可以视为一个字典，几乎所有的sklearn数据集均可以使用data，target，feature_names，DESCR分别获取数据集的数据，标签，特征名称和描述信息。

```
cancer_names = cancer['feature_names']          #取出数据集的特征名
print('breast_cancer数据集的特征名为：\n',cancer_names)
```

```
breast_cancer数据集的特征名为：
 ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
 'mean smoothness' 'mean compactness' 'mean concavity'
 'mean concave points' 'mean symmetry' 'mean fractal dimension'
 'radius error' 'texture error' 'perimeter error' 'area error'
 'smoothness error' 'compactness error' 'concavity error'
 'concave points error' 'symmetry error' 'fractal dimension error'
 'worst radius' 'worst texture' 'worst perimeter' 'worst area'
 'worst smoothness' 'worst compactness' 'worst concavity'
 'worst concave points' 'worst symmetry' 'worst fractal dimension']
```


雨课堂 Rain Classroom

加载datasets模块中数据集

datasets模块常用数据集加载函数及其解释

- 加载后的数据集可以视为一个字典，几乎所有的sklearn数据集均可以使用data，target，feature_names，**DESCR**分别获取数据集的数据，标签，特征名称和描述信息。

```
cancer_desc = cancer['DESCR']                    # 取出数据集的描述信息
print('breast_cancer数据集的描述信息为：\n', cancer_desc)
```

```
breast_cancer数据集的描述信息为：
 Breast Cancer Wisconsin (Diagnostic) Database
=============================================

Notes
-----
Data Set Characteristics:
    :Number of Instances: 569

    :Number of Attributes: 30 numeric, predictive attributes and the class

    :Attribute Information:
        - radius (mean of distances from center to points on the perimeter)
        - texture (standard deviation of gray-scale values)
```

32

将数据集划分为训练集和测试集

常用划分方式

- 在数据分析过程中，为了保证模型在实际系统中能够起到预期作用，一般需要将样本分成独立的三部分：
  - 训练集(**train set**)：用于**构建模型**。
  - 验证集(**validation set**)：用于确定网络结构(**评估模型**)或者控制模型复杂程度的参数(**超参数**)。
  - 测试集（test set）：用于检验选定的最优的模型的性能(**评估最终模型的泛化能力**)。
- 典型的划分方式是训练集占总样本的50%，而验证集和测试集各占25%。
- 当数据总量较少的时候，使用上面的方法将数据划分为三部分就不合适了
  - 提出**K折交叉验证法**

33

将数据集划分为训练集和测试集

K折交叉验证法

- 留少部分做测试集，然后对其余N个样本采用**K折交叉验证法**，基本步骤如下：
  - 将样本打乱，均匀分成K份。
  - 轮流选择其中K - 1份做训练，剩余的一份做验证。
  - 计算预测误差平方和，把K次的预测误差平方和的均值作为选择最优模型结构的依据。

- **sklearn的model_selection模块提供了train_test_split函数，能够对数据集进行拆分**

雨课堂 Rain Classroom

将数据集划分为训练集和测试集

train_test_split函数

- sklearn的model_selection模块提供了train_test_split函数，能够对数据集进行拆分，其格式如下

*sklearn.model_selection.train_test_split(*arrays, **options)*

参数名称	说明
*arrays	接收一个或多个数据集。代表需要划分的数据集，若为分类回归则分别传入数据和标签，若为聚类则传入数据。无默认。
test_size	接收float，int，None类型的数据。代表测试集的大小。如果传入的为float类型的数据则需要限定在0-1之间，代表测试集在总数中的占比；如果传入为int类型的数据，则表示测试集记录的绝对数目。该参数与train_size可以只传入一个。在0.21版本前，若test_size和train_size均为默认则testsize为25%。
train_size	接收float，int，None类型的数据。代表训练集的大小。该参数与test_size可以只传入一个。
random_state	接收int。代表随机种子编号，相同随机种子编号产生相同的随机结果，不同的随机种子编号产生不同的随机结果。默认为None。
shuffle	接收boolean。代表是否进行有放回抽样。若该参数取值为True则stratify参数必须不能为空。
stratify	接收array或者None。如果不为None，则使用传入的标签进行分层抽样。

雨课堂 Rain Classroom

将数据集划分为训练集和测试集

train_test_split函数

- train_test_split函数根据传入的数据，分别将传入的数据划分为训练集和测试集。
  - 如果传入的是1组数据（没有标记的纯数据特征），那么生成的就是这一组数据随机划分后的训练集和测试集，总共2组。
  - 如果传入的是2组数据（1组数据特征+1组标记），则生成的训练集和测试集分别2组，总共4组。

- train_test_split是最常用的数据划分方法，在model_selection模块中还提供了其他数据集划分的函数，如PredefinedSplit，ShuffleSplit等。

雨课堂 Rain Classroom

将数据集划分为训练集和测试集

train_test_split函数

```
print('原始数据集数据的形状为：', cancer_data.shape)
print('原始数据集标签的形状为：', cancer_target.shape)
from sklearn.model_selection import train_test_split
cancer_data_train, cancer_data_test, cancer_target_train, cancer_target_test = \
        train_test_split(cancer_data, cancer_target, test_size=0.2, random_state=42)
print('训练集数据的形状为：', cancer_data_train.shape)
print('测试集数据的形状为：', cancer_data_test.shape)
print('训练集标签的形状为：', cancer_target_train.shape)
print('测试集标签的形状为：', cancer_target_test.shape)
```

```
原始数据集数据的形状为：  (569, 30)
原始数据集标签的形状为：  (569,)
训练集数据的形状为：  (455, 30)
测试集数据的形状为：  (114, 30)
训练集标签的形状为：  (455,)
测试集标签的形状为：  (114,)
```


雨课堂 Rain Classroom

sklearn拟合(训练)和预测

estimator估算器基础

- Scikit-learn提供了**数十种内置的机器学习算法和模型**，称为估算器（estimator）。
- 每个估算器可以使用它的拟合方法与样本数据拟合。
- 例如：使用一些非常基本的数据来训练一个随机森林分类器

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> model = RandomForestClassifier(random_state=0)        #选取随机森林估算器
>>> model.fit(cancer_data_train, cancer_target_train)     #拟合样本数据（训练模型）
```

- 估算器拟合后，可用于预测新数据的目标值。

```
>>> model.predict(cancer_data_test)   # 预测数据分类（预测2条训练数据的标签）
array([0, 1，0, 1，1，0，.....])
```

38

雨课堂 Rain Classroom

sklearn机器学习工作流程

- 机器学习工作流程通常由不同的部分组成。
- 典型流程包括：
  - 数据预处理、转换（转换器——.fit()， .transform()）
  - 模型训练（估算器——.fit()）
  - 模型评估（交叉验证工具——cross_val_score(model, X, y, cv).mean()）
  - 使用模型进行数据预测（估算器——.predict()）
  - 对模型的效果进行评价（不同模型有不同的评价指标，如准确率 accuracy_score(test,testlabel)）

雨课堂 Rain Classroom

使用sklearn转换器进行数据预处理与降维

transformer（转换器）

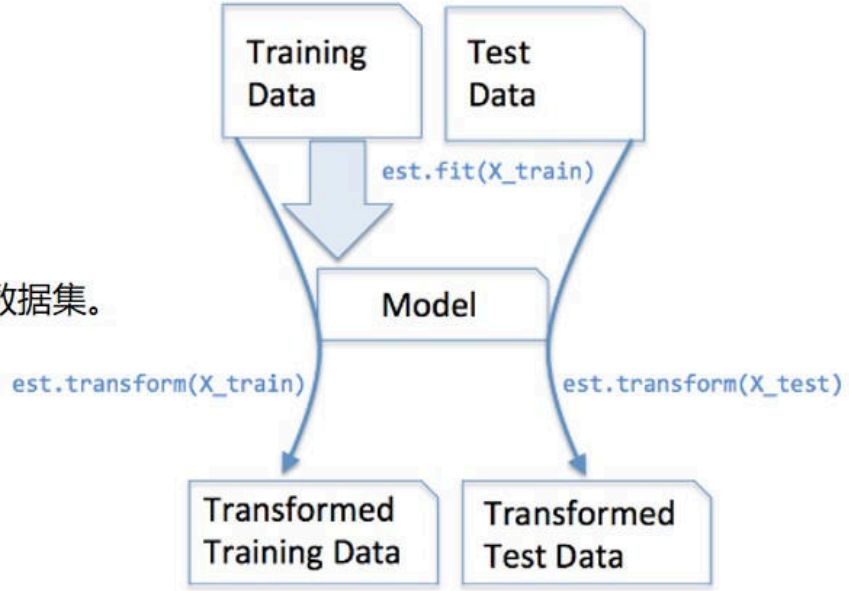
- 为了帮助用户实现大量的特征处理相关操作，sklearn把相关的功能封装为**转换器(transformer)**。
- 在sklearn中，**预处理器和转换器**遵循与estimator对象相同的API
- 使用sklearn转换器能够实现对传入的NumPy数组进行：
  - 标准化处理，归一化处理，二值化处理，PCA降维等操作
- **转换器主要包括三个方法：fit、transform、fit_transform**

雨课堂 Rain Classroom

使用sklearn转换器进行数据预处理与降维

transformer（转换器）

- fit()：
  - 用于从训练数据生成转换模型参数
- transform()：
  - 将从fit()方法生成的参数，应用于模型以生成转换数据集。
- fit_transform()：
  - 组合fit()和transform()的api



使用sklearn转换器进行数据预处理与降维

transformer（转换器）

- 转换器主要包括三个方法：fit、transform、fit_transform

方法名称	说明
fit	fit方法主要通过分析特征和目标值，提取有价值的信息，这些信息可以是统计量，也可以是权值系数等。
transform	transform方法主要用来对特征进行转换。从可利用信息的角度可分为无信息转换和有信息转换。无信息转换是指不利用任何其他信息进行转换，比如指数和对数函数转换等。有信息转换根据是否利用目标值向量又可分为无监督转换和有监督转换。无监督转换指只利用特征的统计信息的转换，比如标准化和PCA降维等。有监督转换指既利用了特征信息又利用了目标值信息的转换，比如通过模型选择特征和LDA降维等。
fit_transform	fit_transform方法就是先调用fit方法，然后调用transform方法。

使用sklearn转换器进行数据预处理与降维

transformer（转换器）—— 实例

- 在数据分析过程中，各类特征处理相关的操作都需要对训练集和测试集分开操作，需要将训练集的操作规则，权重系数等应用到测试集中。
- 在第五章介绍pandas的时候，我们使用pandas库进行标准化处理等，应用至测试集的过程相对繁琐，使用sklearn转换器可以解决这一困扰。

$$X^* = \frac{X - \min}{\max - \min}$$

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler                    #离差标准化
Scaler = MinMaxScaler().fit(cancer_data_train)                    #生成规则
cancer_trainScaler = Scaler.transform(cancer_data_train)          #将规则应用于训练集
cancer_testScaler = Scaler.transform(cancer_data_test)            #将规则应用于测试集
```


雨课堂 Rain Classroom

使用sklearn转换器进行数据预处理与降维

transformer（转换器）—— 实例

```
print('离差标准化前训练集数据的最小值为：',np.min(cancer_data_train))
print('离差标准化后训练集数据的最小值为：',np.min(cancer_trainScaler))
print('离差标准化前训练集数据的最大值为：',np.max(cancer_data_train))
print('离差标准化后训练集数据的最大值为：',np.max(cancer_trainScaler))
print('离差标准化前测试集数据的最小值为：',np.min(cancer_data_test))
print('离差标准化后测试集数据的最小值为：',np.min(cancer_testScaler))
print('离差标准化前测试集数据的最大值为：',np.max(cancer_data_test))
print('离差标准化后测试集数据的最大值为：',np.max(cancer_testScaler))
```

```
0.0
0.0
4254.0
1.0000000000000002
0.0
-0.057127602776294695
3432.0
1.3264399566986453
```

- 通过代码运行发现，离差标准化之后的训练数据集数据的最小值、最大值的确限定在[0,1]区间，同时由于测试集应用了训练集的离差标准化规则，数据超出了[0,1]的范围，这也侧面证明了此处应用了训练集的规则。

44

雨课堂 Rain Classroom

使用sklearn转换器进行数据预处理与降维

sklearn部分预处理函数及其作用

- sklearn除了提供离差标准化函数MinMaxScaler外，还提供了一系列数据预处理函数，具体如下表：

函数名称	说明
MinMaxScaler	对特征进行离差标准化。
StandardScaler	对特征进行标准差标准化。
Normalizer	对特征进行归一化。
Binarizer	对定量特征进行二值化处理。
OneHotEncoder	对定性特征进行独热编码处理。
FunctionTransformer	对特征进行自定义函数变换。

- 更多参考sklearn官网 http://scikit-learn.org/stable/modules/classes.html#module-sklearn.preprocessing

使用sklearn转换器进行数据预处理与降维

PCA降维算法函数（主成分分析）

- sklearn除了提供基本的特征变换函数外，还提供了降维算法，特征选择算法，这些算法的使用也是通过转换器的方式进行的。

```
from sklearn.decomposition import PCA
pca_model = PCA(n_components=10).fit(cancer_trainScaler)   #生成规则
cancer_trainPca = pca_model.transform(cancer_trainScaler)   #将规则应用于训练集
cancer_testPca = pca_model.transform(cancer_testScaler)   #将规则应用于测试集
print('PCA降维前训练集数据的形状为：',cancer_trainScaler.shape)
print('PCA降维后训练集数据的形状为：',cancer_trainPca.shape)
print('PCA降维前测试集数据的形状为：',cancer_testScaler.shape)
print('PCA降维后测试集数据的形状为：',cancer_testPca.shape)
```

使用sklearn转换器进行数据预处理与降维

PCA降维算法函数（主成分分析）

- sklearn除了提供基本的特征变换函数外，还提供了降维算法，特征选择算法，这些算法的使用也是通过转换器的方式进行的。

```
from sklearn.decomposition import PCA
pca_model = PCA(n_components=10).fit(cancer_trainScaler)   #生成规则
cancer_trainPca = pca_model.transform(cancer_trainScaler)   #将规则应用于训练集
cancer_testPca = pca_model.transform(cancer_testScaler)   #将规则应用于测试集
print('PCA降维前训练集数据的形状为：',cancer_trainScaler.shape)
print('PCA降维后训练集数据的形状为：',cancer_trainPca.shape)
print('PCA降维前测试集数据的形状为：',cancer_testScaler.shape)
print('PCA降维后测试集数据的形状为：',cancer_testPca.shape)
```

```
PCA降维前训练集数据的形状为： (455, 30)
PCA降维后训练集数据的形状为： (455, 10)
PCA降维前测试集数据的形状为： (114, 30)
PCA降维后测试集数据的形状为： (114, 10)
```

47

使用sklearn转换器进行数据预处理与降维

PCA降维算法函数常用参数及其作用

函数名称	说明
n_components	接收None，int，float或string。未指定时，代表所有特征均会被保留下来；如果为int，则表示将原始数据降低到n个维度；如果为float，同时svd_solver参数等于full；赋值为string，比如n_components='mle'，将自动选取特征个数n，使得满足所要求的方差百分比。默认为None。
copy	接收bool。代表是否在运行算法时将原始数据复制一份，如果为True，则运行后，原始数据的值不会有任何改变；如果为False，则运行PCA算法后，原始训练数据的值会发生改变。默认为True
whiten	接收boolean。表示白化，所谓白化，就是对降维后的数据的每个特征进行归一化，让方差都为1。默认为False。
svd_solver	接收string｛‘auto’，‘full’，‘arpack’，‘randomized’｝。代表使用的SVD算法。randomized一般适用于数据量大，数据维度多，同时主成分数目比例又较低的PCA降维，它使用了一些加快SVD的随机算法。full是使用SciPy库实现的传统SVD算法。arpack和randomized的适用场景类似，区别是randomized使用的是sklearn自己的SVD实现，而arpack直接使用了SciPy库的sparse SVD实现。auto则代表PCA类会自动在上述三种算法中去权衡，选择一个合适的SVD算法来降维。默认为auto。

雨课堂 Rain Classroom

PCA（主成分分析）降维算法（补充）

PCA降维算法几何含义

- PCA 是一种基于多变量的降维技术，旨在找到数据中的主成分，并利用这些主成分表征原始数据，从而达到降维的目的 。主要用于降维、可视化、去相关、分类、确定潜在因子、压缩和去噪音等方面。是特征工程中不可缺少的一部分，有利于我们构建更加适合模型的特征。
- **PCA的主要思想是将n维特征映射到k维上，这k维是全新的正交特征也被称为主成分，是在原有n维特征的基础上重新构造出来的k维特征。**
  - 第一个新坐标轴选择是原始数据中方差最大的方向；
  - 第二个新坐标轴选取是与第一个坐标轴正交的平面中使得方差最大的；
  - 第三个轴是与第1,2个轴正交的平面中方差最大的。
  - 依次类推，可以得到n个这样的坐标轴。
- **我们发现，大部分方差都包含在前面k个坐标轴中，后面的坐标轴所含的方差几乎为0。于是，我们可以忽略余下的坐标轴，只保留前面k个含有绝大部分方差的坐标轴。**

49

雨课堂 Rain Classroom

PCA（主成分分析）降维算法（补充）

PCA降维算法求解步骤

- 设有 m 条 n 维数据。

1. 将原始数据按列组成 n 行 m 列矩阵 X；
2. 将 X 的每一行进行零均值化，即减去这一行的均值；
3. 求出协方差矩阵：$C = \frac{1}{m} X X^{\mathsf{T}}$
4. 求出协方差矩阵的特征值及对应的特征向量；
5. 将特征向量按对应特征值大小从上到下按行排列成矩阵，取前 k 行组成矩阵 P；
6. ***Y = PX*** 即为降维到 k 维后的数据。

雨课堂 Rain Classroom

PCA（主成分分析）降维算法（补充）

PCA降维算法性质

- **缓解维度灾难**：PCA 算法通过舍去一部分信息之后能使得样本的采样密度增大（因为维数降低了），这是缓解维度灾难的重要手段；
- **降噪：**当数据受到噪声影响时，最小特征值对应的特征向量往往与噪声有关，将它们舍弃能在一定程度上起到降噪的效果；
- **过拟合：**PCA 保留了主要信息，但这个主要信息只是针对训练集的，而且这个主要信息未必是重要信息。有可能舍弃了一些看似无用的信息，但是这些看似无用的信息恰好是重要信息，只是在训练集上没有很大的表现，所以 PCA 也可能加剧了过拟合；
- **特征独立：**PCA 不仅将数据压缩到低维，它也使得降维之后的数据各特征相互独立；

51

雨课堂 Rain Classroom

如果不指定降维数量，则所有维数都保留，可以返回所有的特征向量

```
In [13]: 1 pca_model = PCA().fit(cancer_trainScaler)    #生成规则

In [15]: 1 pca_model.components_        #返回模型的各个特征向量
```

```
-3.60893344e-02,  1.08547151e-01, -1.49306677e-01,
 4.70891703e-01, -4.03569889e-01,  4.79317798e-03,
-4.98247099e-03, -3.89714786e-01,  8.99700401e-03,
 3.47239124e-01, -3.25454079e-02,  1.92368013e-03,
 7.44058868e-02, -2.50099626e-01, -7.77030301e-03,
 4.21715077e-03,  3.32104823e-02, -1.37649345e-01,
-4.75532056e-02,  3.63583466e-01, -3.31674462e-02,
-1.45314397e-02, -3.38140812e-02, -2.25593623e-01,
 1.48804094e-01,  4.19485842e-03,  2.25196035e-02],
[ 1.68351172e-01, -3.76730128e-02,  7.81913533e-02,
-2.18984084e-01,  5.82486123e-04, -1.25057525e-01,
 3.71613243e-01, -2.70517779e-01, -1.80989585e-03,
 9.65105775e-02,  3.44190205e-01, -1.26635207e-02,
-4.27919564e-01,  1.04499019e-01, -1.45468374e-02,
 8.57933759e-02, -4.10119051e-01,  1.54120624e-01,
 2.90754835e-03, -7.01769640e-02,  2.80523058e-01,
 3.46351459e-02, -2.65297669e-01, -6.23715558e-02,
-5.01934782e-03,  5.49728983e-02, -6.80253626e-02,
 1.35922474e-02, -9.05997622e-03, -2.66678103e-02],
[-1.18131088e-01, -2.61531735e-02, -1.40432899e-01,
```

可以返回各个成分各自的方差，和方差百分比，即贡献率，可以根据贡献率决定降维的数量

In [16]:
```
1 pca_model.explained_variance_          #返回各个成分各自的方差
```

Out[16]:
```
array([3.42686834e-01, 1.24359064e-01, 4.82054472e-02, 4.27455813e-02,
       2.66988848e-02, 2.25159037e-02, 1.05312007e-02, 8.90367071e-03,
       6.72568263e-03, 6.57642104e-03, 5.65102011e-03, 5.33668690e-03,
       4.79251249e-03, 3.11587045e-03, 1.93896571e-03, 1.63812458e-03,
       1.21186308e-03, 1.00220100e-03, 9.66163376e-04, 7.17562404e-04,
       6.18594733e-04, 5.21971130e-04, 4.01015010e-04, 3.61701240e-04,
       2.60741468e-04, 2.06257936e-04, 1.24621696e-04, 3.46448162e-05,
       1.83214275e-05, 3.64191971e-06])
```

In [14]:
```
1 pca_model.explained_variance_ratio_          #返回各个成分各自的方差百分比（贡献率）
```

Out[14]:
```
array([5.12336080e-01, 1.85923791e-01, 7.20698533e-02, 6.39070468e-02,
       3.99163336e-02, 3.36625418e-02, 1.57447371e-02, 1.33114882e-02,
       1.00552736e-02, 9.83211914e-03, 8.44859273e-03, 7.97864690e-03,
       7.16507557e-03, 4.65840148e-03, 2.89886273e-03, 2.44908834e-03,
       1.81180342e-03, 1.49834683e-03, 1.44446856e-03, 1.07279613e-03,
       9.24833899e-04, 7.80376181e-04, 5.99539982e-04, 5.40763685e-04,
       3.89823152e-04, 3.08367209e-04, 1.86316441e-04, 5.17959477e-05,
       2.73915640e-05, 5.44487469e-06])
```


雨课堂 Rain Classroom

sklearn转换器任务实现（自我练习）

1.读取数据

- 获取sklearn自带的boston数据集

2.将数据集划分为训练集和测试集

- 使用train_test_split划分boston数据集

3.使用转换器进行数据预处理

- 使用stdScale.transform分别对训练集和测试集进行预处理

4.使用转换器进行PCA降维

- 使用PCA.transform分别对训练集和测试集进行PCA降维

雨课堂 Rain Classroom

sklearn转换器任务实现（自我练习）

1.读取数据

- 获取sklearn自带的boston数据集

```
from sklearn.datasets import load_boston
boston = load_boston()
boston_data = boston['data']
boston_target = boston['target']
boston_names = boston['feature_names']
print('boston数据集数据的形状为：',boston_data.shape)
print('boston数据集标签的形状为：',boston_target.shape)
print('boston数据集特征名的形状为：',boston_names.shape)
```

```
boston数据集数据的形状为：  (506, 13)
boston数据集标签的形状为：  (506,)
boston数据集特征名的形状为：  (13,)
```

55

sklearn转换器任务实现（自我练习）

1.读取数据

- 获取sklearn自带的boston数据集
  - CRIM - 城镇人均犯罪率
  - ZN - 住宅用地超过 25000 sq.ft. 的比例.
  - INDUS - 城镇非零售商用土地的比例.
  - CHAS - 查理斯河哑变量（如果边界是河流，则为1；否则为0）
  - NOX - 一氧化氮浓度 (parts per 10 million)
  - RM - 住宅平均房间数
  - AGE - 1940 年之前建成的自用房屋比例
  - DIS - 到波士顿五个中心区域的加权距离
  - RAD - 辐射性公路的接近指数
  - TAX - 每 10000 美元的全值财产税率
  - PTRATIO - 城镇师生比例
  - B - 1000(Bk - 0.63)^2，其中 Bk 指代城镇中黑人的比例
  - LSTAT - 人口中地位低下者的比例
  - MEDV - 自住房的平均房价，以千美元计

sklearn转换器任务实现（自我练习）

2.将数据集划分为训练集和测试集

- 使用train_test_split划分boston数据集

```
from sklearn.model_selection import train_test_split
boston_data_train, boston_data_test, boston_target_train, boston_target_test = \
    train_test_split(boston_data, boston_target, test_size=0.2, random_state=42)
print('训练集数据的形状为：',boston_data_train.shape)
print('训练集标签的形状为：',boston_target_train.shape)
print('测试集数据的形状为：',boston_data_test.shape)
print('测试集标签的形状为：',boston_target_test.shape)
```

```
训练集数据的形状为： (404, 13)
训练集标签的形状为： (404,)
测试集数据的形状为： (102, 13)
测试集标签的形状为： (102,)
```

57

sklearn转换器任务实现（自我练习）

3.使用转换器进行数据预处理

- 使用stdScale.transform分别对训练集和测试集进行预处理

```
import numpy as np
from sklearn.preprocessing import StandardScaler
stdScale = StandardScaler().fit(boston_data_train)          # 生成规则（标准差标准化）
boston_trainScaler = stdScale.transform(boston_data_train)      # 将规则应用于训练集
boston_testScaler = stdScale.transform(boston_data_test)        # 将规则应用于测试集
print('标准差标准化后训练集数据的方差为：',np.var(boston_trainScaler))
print('标准差标准化后训练集数据的均值为：',np.mean(boston_trainScaler))
print('标准差标准化后测试集数据的方差为：',np.var(boston_testScaler))
print('标准差标准化后测试集数据的均值为：',np.mean(boston_testScaler))
```

58

sklearn转换器任务实现（自我练习）

4.使用转换器进行PCA降维

- 使用PCA.transform分别对训练集和测试集进行PCA降维

```
from sklearn.decomposition import PCA
pca = PCA(n_components=5).fit(boston_trainScaler)    # 生成规则
boston_trainPca = pca.transform(boston_trainScaler)  # 将规则应用于训练集
boston_testPca = pca.transform(boston_testScaler)    # 将规则应用于测试集
print('降维后boston数据集数据测试集的形状为：', boston_trainPca.shape)
print('降维后boston数据集数据训练集的形状为：', boston_testPca.shape)
```

```
降维后boston数据集数据测试集的形状为： (404, 5)
降维后boston数据集数据训练集的形状为： (102, 5)
```


雨课堂 Rain Classroom

目录


1
使用sklearn转换器处理数据
2
构建并评价聚类模型
3
构建并评价分类模型
4
构建并评价回归模型
5
小结

构建并评价聚类模型

任务描述

- **聚类分析**是在没有给定划分类别的情况下，根据数据相似度进行样本分组的一种方法。
- 聚类模型可以将无类标记的数据聚集为多个簇，视为一类，是一种无监督的学习算法。
- 在商业上，聚类可以帮助市场分析人员从消费者数据库中区分出不同的消费群体，并且概况出每一类消费者的消费模式或消费习惯。
- 同时，聚类分析也可以作为数据分析算法中其他分析算法的一个预处理步骤，比如异常值识别、连续性特征离散化等。

任务分析

（1）使用**sklearn估算器**构建K-Means聚类模型

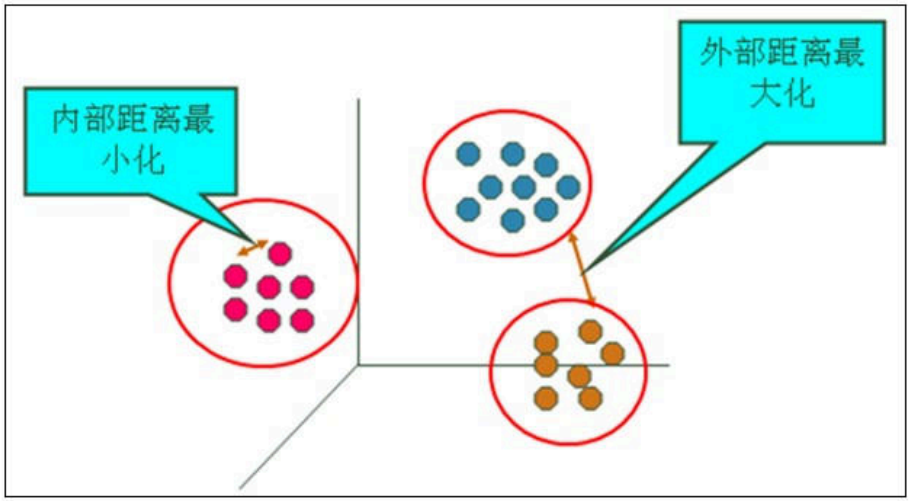
（2）根据**聚类模型评价指标**评价K-Means聚类模型

61

使用sklearn估算器构建聚类模型

聚类

- 聚类的输入是一组未被标记的样本，聚类根据数据自身的距离或相似度将他们划分为若干组；
- 划分的原则是组内（内部）样本距离最小化，而组间（外部）距离最大化，如图所示。



62

使用sklearn估算器构建聚类模型

聚类方法类别

- 常用的聚类算法如表：

算法类别	包括的主要算法
划分(分裂)方法	K-Means算法(K-平均），K-MEDOIDS算法(K-中心点）和CLARANS算法(基于选择的算法）。
层次分析方法	BIRCH算法(平衡迭代规约和聚类），CURE算法(代表点聚类）和CHAMELEON算法(动态模型）。
基于密度的方法	DBSCAN算法(基于高密度连接区域），DENCLUE算法(密度分布函数）和OPTICS算法(对象排序识别）。
基于网格的方法	STING算法(统计信息网络），CLIOUE算法(聚类高维空间）和WAVE-CLUSTER算法(小波变换）。

雨课堂 Rain Classroom

使用sklearn估算器构建聚类模型

cluster提供的聚类算法及其适用范围

- sklearn常用的聚类算法模块cluster提供的聚类算法及其适用范围如下所示：

函数名称	参数	适用范围(样本量、聚类数目)	距离度量
KMeans	簇数	样本量很大，聚类数目中等	点之间的距离
Spectral clustering	簇数	样本量中等，聚类数目较小	图距离
Ward hierarchical clustering	簇数	样本量较大，聚类数目较大	点之间的距离
Agglomerative clustering	簇数，链接类型，距离	样本量较大，聚类数目较大	任意成对点线图间的距离
DBSCAN	半径大小，最低成员数目	样本量很大，聚类数目中等	最近的点之间的距离
Birch	分支因子，阈值， 可选全局集群	样本量很大，聚类数目较大	点之间的欧式距离

- **详情参考：https://sklearn.org/modules/clustering.html#clustering**

64

了解K-Means聚类算法

1. 基本概念

K-Means聚类算法是一种基于质心的划分方法，输入聚类个数k，以及包含n个数据对象的数据库，输出满足误差平方和最小标准的k个聚类。算法步骤如下。

① 从n个样本数据中随机选取k个对象作为初始的聚类中心。

② 分别计算每个样本到各个聚类质心的距离，将样本分配到距离最近的那个聚类中心类别中。

③ 所有样本分配完成后，重新计算k个聚类的中心。

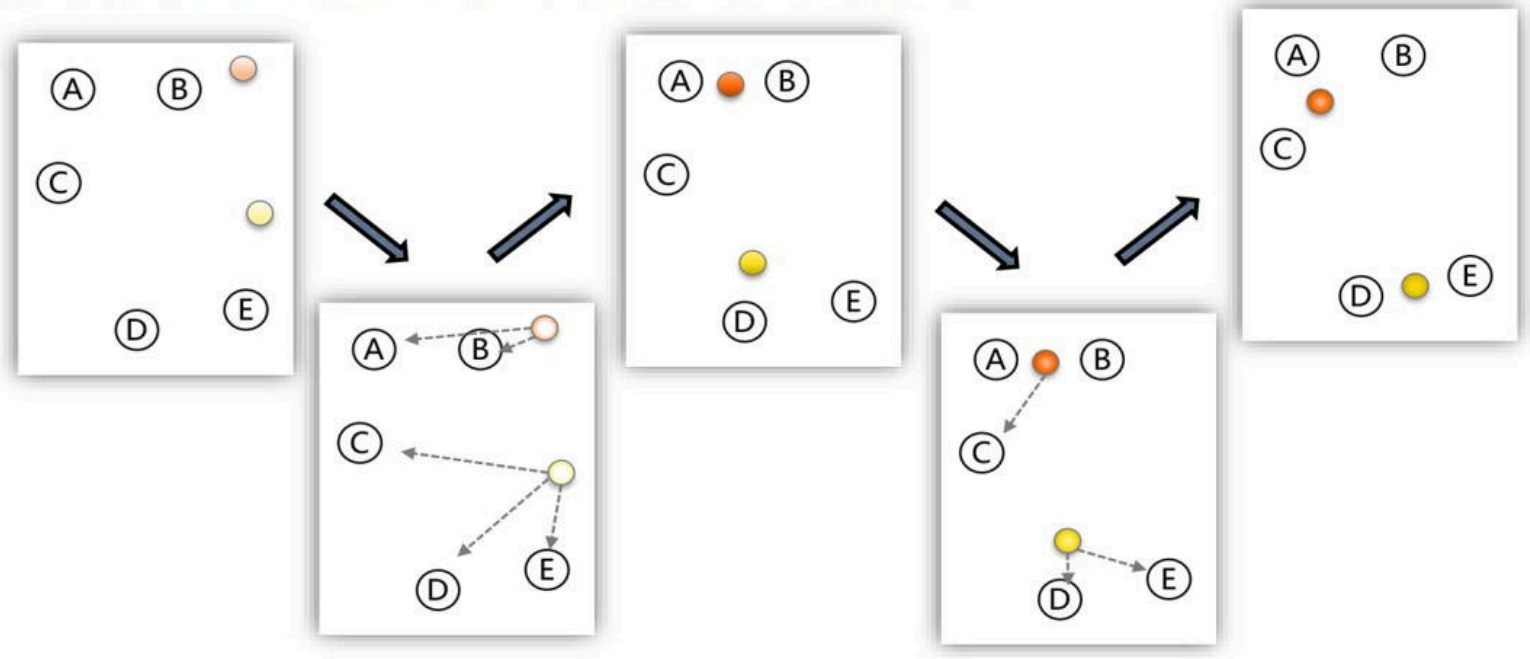
④ 与前一次计算得到的k个聚类中心比较，如果聚类中心发生变化，转❷，否则转❺。

⑤ 当质心不发生变化时停止并输出聚类结果。

了解K-Means聚类算法

1. 基本概念

K-Means聚类算法是一种基于质心的划分方法，输入聚类个数k，以及包含n个数据对象的数据库，输出满足误差平方和最小标准的k个聚类。算法步骤如下。



了解K-Means聚类算法

1. 基本概念

- **K-Means聚类算法的结果依赖于初始聚类中心的随机选择。**
- 在实践中，为了得到较好的结果，通常以不同的初始聚类中心多次运行K-Means算法。在所有样本分配完成后，重新计算k个聚类中心时，对于连续数据，聚类中心取该簇的均值。

- K-Means聚类算法的**优点**：原理简单、容易实现，计算复杂度较小。
- **缺点**也较为明显：过分依赖初始值的选择，一旦初始值选择的不好，可能无法得到有效的聚类效果；需要预先给定k值，这往往是最困难的一步；对离群点非常敏感，容易使质心偏移。

了解K-Means聚类算法

2. 数据类型

K-Means聚类算法是在数值类型数据的基础上进行研究，然而数据分析的样本复杂多样，因此要求不仅能够对特征为数值类型的数据进行分析，还要适应数据类型的变化，对不同特征做不同变换，以满足算法的要求。

对于K-means算法，首先要注意的是k值的选择，一般来说，我们会根据对数据的先验经验选择一个合适的k值，如果没有什么先验知识，则可以通过交叉验证选择一个合适的k值。

在确定了k的个数后，我们需要选择k个初始化的质心，就像上图b中的随机质心。由于我们是启发式方法，k个初始化的质心的位置选择对最后的聚类结果和运行时间都有很大的影响，因此需要选择合适的k个质心，最好这些质心不能太近。

雨课堂 Rain Classroom

了解K-Means聚类算法

3. kmeans函数及其参数介绍

- sklearn的cluster模块提供了kmeans函数来构建K-Means聚类模型，其基本语法如下：

```
class sklearn.cluster.KMeans(n_clusters=8, init='k-means++', n_init=10, max_iter=300, tol=0.0001,
                             precompute_distances='auto', verbose=0, random_state=None,
                             copy_x=True, n_jobs=1, algorithm='auto')
```

- 常用参数及其说明如表所示。

参数名称	说 明
n_clusters	Int，可选，默认值：8，要形成的簇数以及要生成的质心数。
init	初始化方法，默认为 'k-means++' ，即以智能方式选择初始聚类中心，以加速收敛。还可以选择 'random' 和 'ndarray'
n_int	接收int，默认10。使用不同质心种子运行k-means算法的次数。最终结果将是n_init连续运行的最佳输出。。
max_iter	单次运行的k-means算法的最大迭代次数。默认300
tol=0.0001	接收float，表示算法收敛的阈值，默认时0.0001

了解K-Means聚类算法

3. kmeans函数及其参数介绍

- sklearn的cluster模块提供了kmeans函数来构建K-Means聚类模型，其基本语法如下：

```
class sklearn.cluster.KMeans(n_clusters=8, init='k-means++', n_init=10, max_iter=300, tol=0.0001,
                             precompute_distances='auto', verbose=0, random_state=None,
                             copy_x=True, n_jobs=1, algorithm='auto')
```

- 常用参数及其说明如表所示。

参数名称	说 明
precompute_distances	接收boolean或auto，表示是否提前计算好样本之间的距离。
verbose	接收int，缺省0，表示不输出日志信息，1表示每隔一段时间打印一次日志，如果大于1，则打印日志更频繁。
random_state	接收int，表示随机数生成器的种子。默认为none
copy_x	boolean，缺省为True，不修改原始数据
n_jobs	接收int，表示任务使用的CPU数量，默认为1
algorithm	“auto”，“full” or “elkan”，default=”auto”

了解DBSCAN聚类算法

1. 基本概念

- DBSCAN(Density-Based Spatial Clustering of Applications with Noise)，具有噪声的基于密度的聚类方法
- 是一种很典型的密度聚类算法；
- K-Means，BIRCH这些一般只适用于凸样本集的聚类
  - 某数据集D中任意两点的连线上的点，也会在数据集D内，那么数据集D就是一个凸集
- DBSCAN既可以适用于凸样本集，也可以适用于非凸样本集。

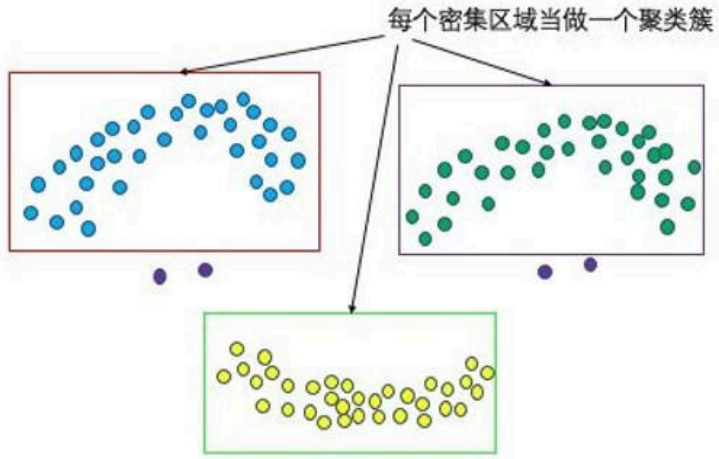
雨课堂 Rain Classroom

了解DBSCAN聚类算法

1. 基本概念

1个核心思想：**基于密度**

- 直观效果上看，DBSCAN算法可以找到样本点的全部密集区域，并把这些密集区域当做一个一个的聚类簇。



DBSCAN的1个核心思想

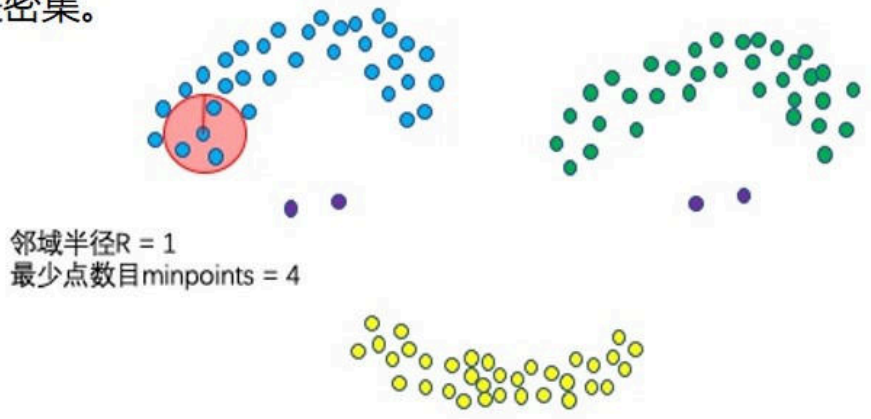
雨课堂 Rain Classroom

了解DBSCAN聚类算法

1. 基本概念

2个算法参数:

- **邻域半径E** 和 **最少点数目**MinPts (Minpoints)
- 这两个算法参数实际可以刻画什么叫密集——当邻域半径R内的点的个数大于最少点数目Minpoints时，就是密集。



DBSCAN中2个算法参数

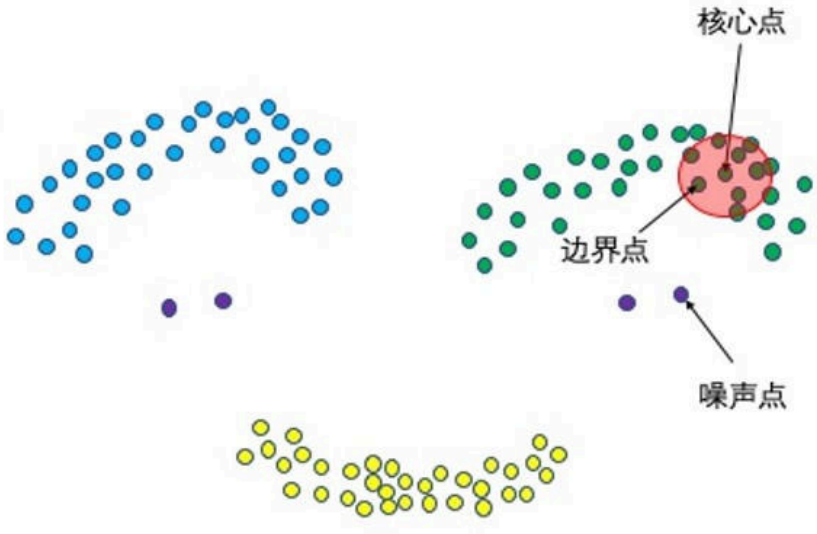
73

了解DBSCAN聚类算法

1. 基本概念

3种点的类别：**核心点，边界点** 和 **噪声点**

- 核心点：在半径E内含有超过MinPts数目的点。
- 边界点：在半径E内点的数量小于MinPts，但是落在核心点的邻域内的点。
- 噪声点：既不是核心点也不是边界点的点。



DBSCAN中3种点的类别

雨课堂 Rain Classroom

了解DBSCAN聚类算法

1. 基本概念

4种点的关系：**密度直达**，**密度可达**，**密度相连**，**非密度相连**

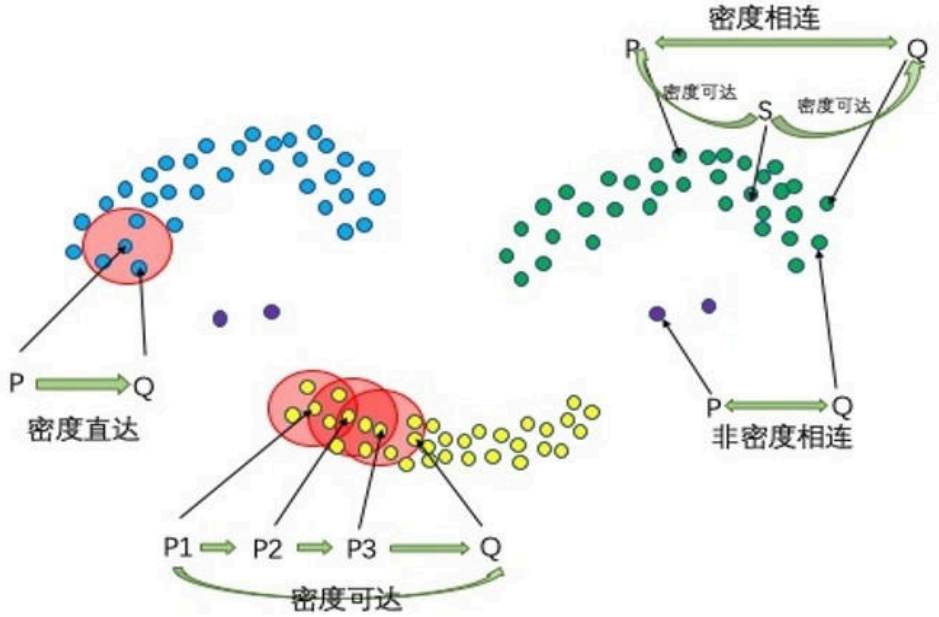
- **密度直达：**如果P为核心点，Q在P的E邻域内，那么称P到Q密度直达。
- **密度可达：**如果存在核心点P2，P3，……，Pn，且P1到P2密度直达，P2到P3密度直达，……，P(n-1)到Pn密度直达，Pn到Q密度直达，则P1到Q密度可达。
- **密度相连：**如果存在核心点S，使得S到P和Q都密度可达，则P和Q密度相连。
- **非密度相连：**如果两个点不属于密度相连关系，则两个点非密度相连。

雨课堂 Rain Classroom

了解DBSCAN聚类算法

1. 基本概念

4种点的关系：**密度直达，密度可达，密度相连，非密度相连**



DBSCAN中4种点的关系

76

了解DBSCAN聚类算法

1. 基本概念

4种点的关系：**密度直达，密度可达，密度相连，非密度相连**

- **密度直达：**任何核心点到其自身密度直达，密度直达不具有对称性，如果P到Q密度直达，那么Q到P不一定密度直达。
- **密度可达：**密度可达也不具有对称性。
- **密度相连：**密度相连具有对称性，如果P和Q密度相连，那么Q和P也一定密度相连。**密度相连的两个点属于同一个聚类簇。**
- **非密度相连：****非密度相连的两个点属于不同的聚类簇**，或者其中存在噪声点。

77

雨课堂 Rain Classroom

了解DBSCAN聚类算法

2. 算法步骤

输入：包含n个对象的数据库，半径E，最少数目MinPts;

输出：所有生成的簇，达到密度要求。

① 检测数据库中尚未检查过的对象p，如果p未被处理(归为某个簇或者标记为噪声)，且p是核心点，则建立新簇C，将p邻域中的所有点加入候选集N；

② 对候选集N 中所有尚未被处理的对象q，检查q是否为核心点，如果是，则将q邻域内所有对象加入N；如果q 未归入任何一个簇，则将q 加入C；

③ 重复步骤❷，继续检查N 中未处理的对象，直至当前候选集N为空；

④ 重复步骤❶~❸，直到所有对象都归入了某个簇或标记为噪声；

78

了解DBSCAN聚类算法

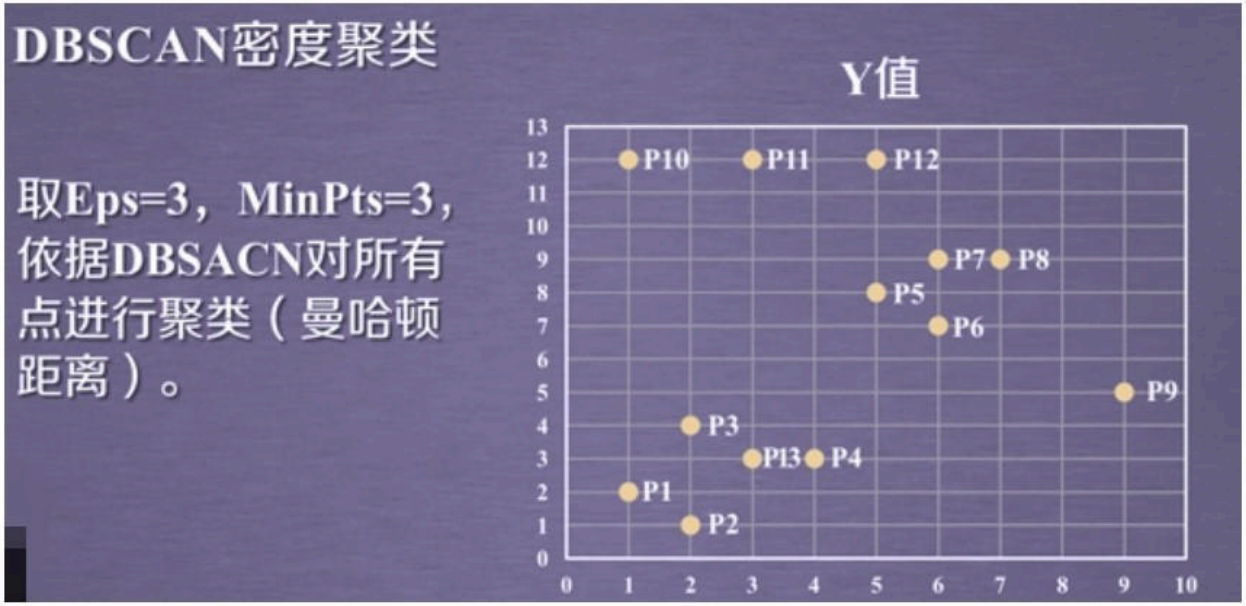
3. 算法实例

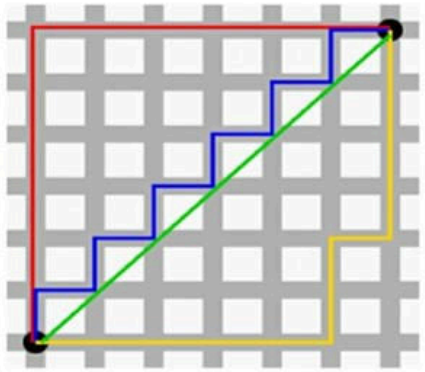
举例：有如下13个样本点，使用DBSCAN进行聚类。

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13
X	1	2	2	4	5	6	6	7	9	1	3	5	3
Y	2	1	4	3	8	7	9	9	5	12	12	12	3

了解DBSCAN聚类算法

3. 算法实例



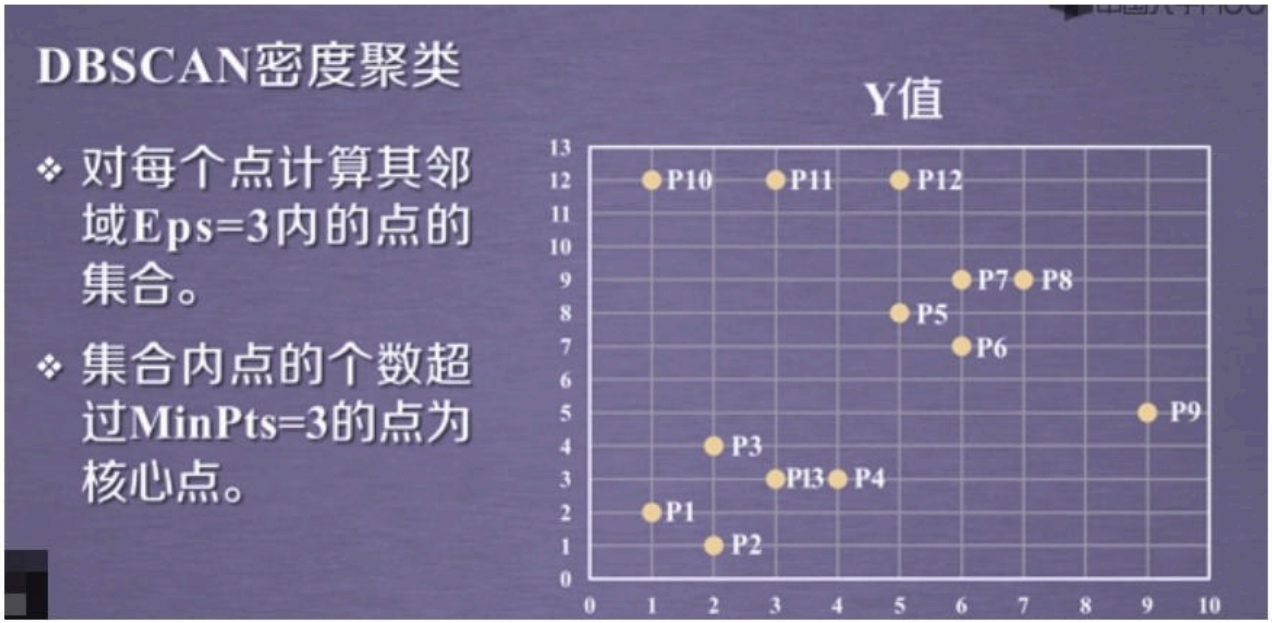


上图中红线代表曼哈顿距离，绿色代表欧氏距离，也就是直线距离，而蓝色和黄色代表等价的曼哈顿距离。

了解DBSCAN聚类算法

3. 算法实例

第一步，顺序扫描数据集的样本点，首先取到 p1(1,2)



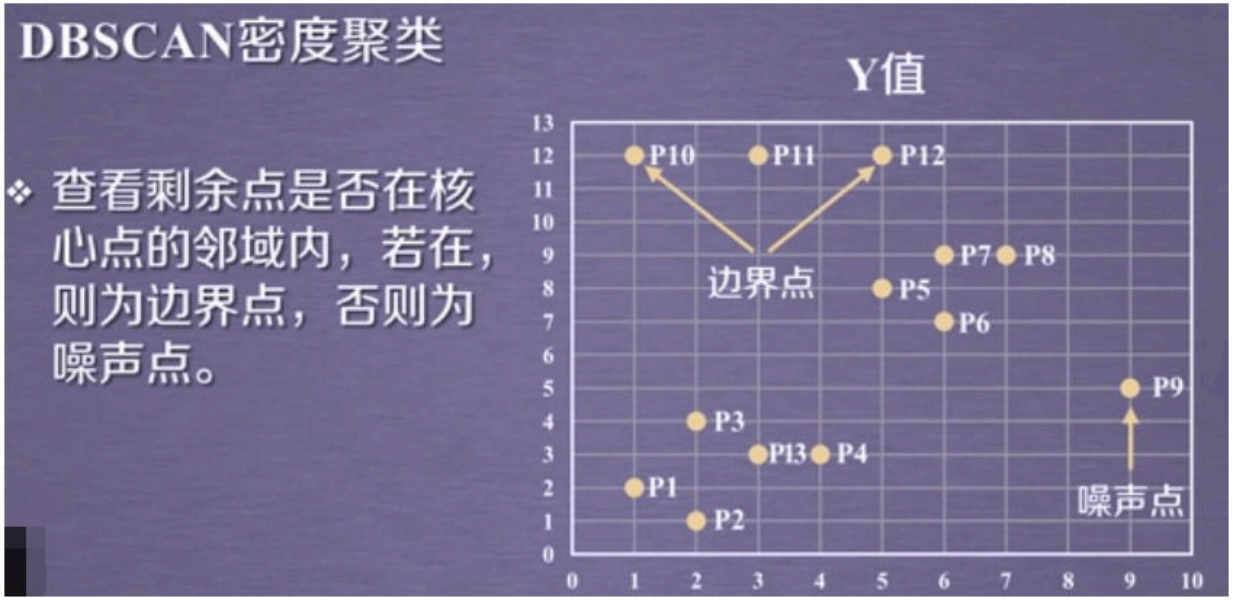
① 计算出 p1 的 Eps 邻域为 {p1,p2,p3,p13},p1是核心点;

② 以 p1 为核心点建立簇 C1，找出所有从 p1 密度可达的点，也加入到C1

了解DBSCAN聚类算法

3. 算法实例

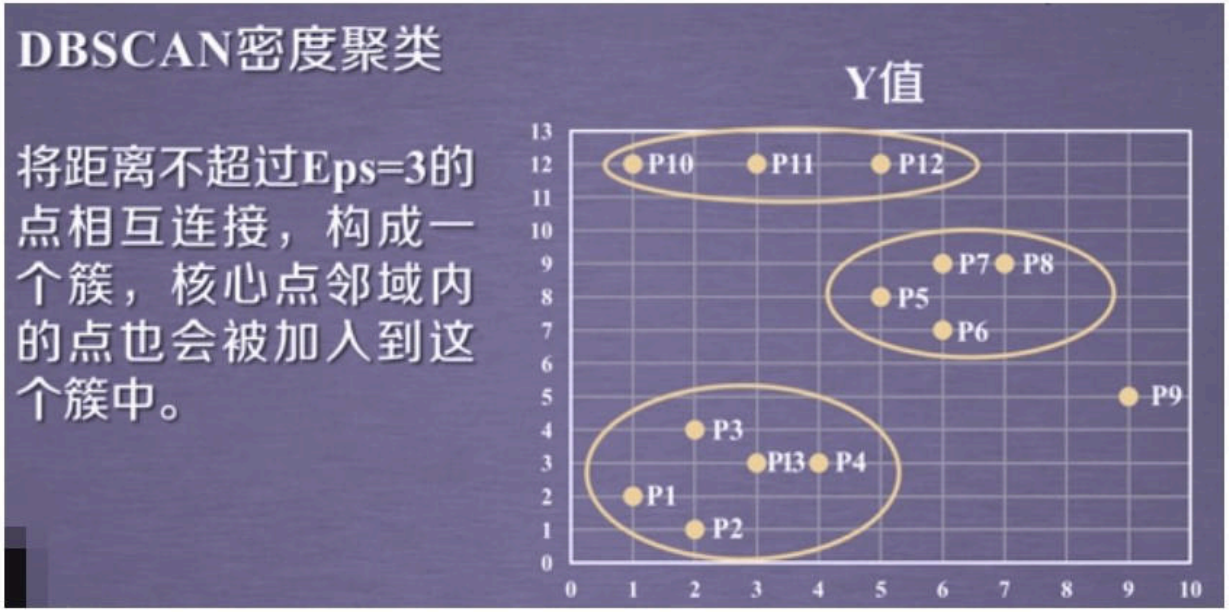
第二步，继续顺序扫描数据集的样本点，直至所有样本点都处理结束。



82

了解DBSCAN聚类算法

3. 算法实例



83

了解DBSCAN聚类算法

4. 算法优缺点

DBSCAN 算法的主要优点如下。

1）可以对任意形状的稠密数据集进行聚类，而 k-means 之类的聚类算法一般只适用于凸数据集。

2）可以在聚类的同时发现异常点，对数据集中的异常点不敏感。

3）聚类结果没有偏倚，而 k-means 之类的聚类算法的初始值对聚类结果有很大影响。

如果数据集是稠密的，并且数据集不是凸的，那么用DBSCAN会比K-Means聚类效果好很多

如果数据集不是稠密的，则不推荐用DBSCAN来聚类。

84

了解DBSCAN聚类算法

4. 算法优缺点

DBSCAN 算法的主要缺点如下。

1）样本集的密度不均匀、聚类间距差相差很大时，聚类质量较差，不适合DBSCAN 算法。

2）样本集较大时，聚类收敛时间较长。

3）调试参数比较复杂时，主要需要对距离阈值 Eps，邻域样本数阈值 MinPts 进行联合调参，不同的参数组合对最后的聚类效果有较大影响。

4）对于整个数据集只采用了一组参数。如果数据集中存在不同密度的簇或者嵌套簇，则 DBSCAN 算法不能处理。为了解决这个问题，有人提出了 OPTICS 算法。

5）DBSCAN 算法可过滤噪声点，这同时也是其缺点，这造成了其不适用于某些领域，如对网络安全领域中恶意攻击的判断。

雨课堂 Rain Classroom

使用sklearn估算器构建聚类模型

sklearn估算器

- 聚类算法实现需要sklearn估算器(estimator)。
- sklearn估算器拥有 fit 和 predict 两个方法。
- 两个方法的作用如下：

方法名称	说明
fit	fit方法主要用于训练算法。该方法可接收用于有监督学习的训练集及其标签两个参数，也可以接收用于无监督学习的数据。
predict	predict用于预测有监督学习的测试集标签，亦可以用于划分传入数据的类别。

使用sklearn估算器构建聚类模型

- 以iris数据为例，使用sklearn估算器构建K-Means聚类模型，代码如下：

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans
iris = load_iris()
iris_data = iris['data']                    # 提取数据集中的特征
iris_target = iris['target']                # 提取数据集中的标签
iris_names = iris['feature_names']          # 提取特征名
```


雨课堂 Rain Classroom

使用sklearn估算器构建聚类模型

```
scale = MinMaxScaler().fit(iris_data)                    # 使用转换器构建并训练标准化规则
iris_dataScale = scale.transform(iris_data)              # 应用规则
#使用估算器构建并训练聚类模型(分3类)
kmeans = KMeans(n_clusters=3,random_state=123).fit(iris_dataScale)
print('构建的K-Means模型为：\n', kmeans)
result = kmeans.predict([[1.5,1.5,1.5,1.5]])             #使用构建的模型进行预测
print('花瓣花萼长度宽度全为1.5的鸢尾花预测类别为：', result[0])
```

```
构建的K-Means模型为：
 KMeans(algorithm='auto', copy_x=True, init='k-means++', max_iter=300,
    n_clusters=3, n_init=10, n_jobs=1, precompute_distances='auto',
    random_state=123, tol=0.0001, verbose=0)
花瓣花萼长度宽度全为1.5的鸢尾花预测类别为： 0
```

88

使用sklearn估算器构建聚类模型

二维空间查看聚类效果 —— TSNE函数降维

- 聚类完成后如果想通过可视化的方式查看聚类效果，可以通过sklearn的manifold模块中的TSNE函数进行降维(如降成两维)，则可以实现二维空间可视化展现。

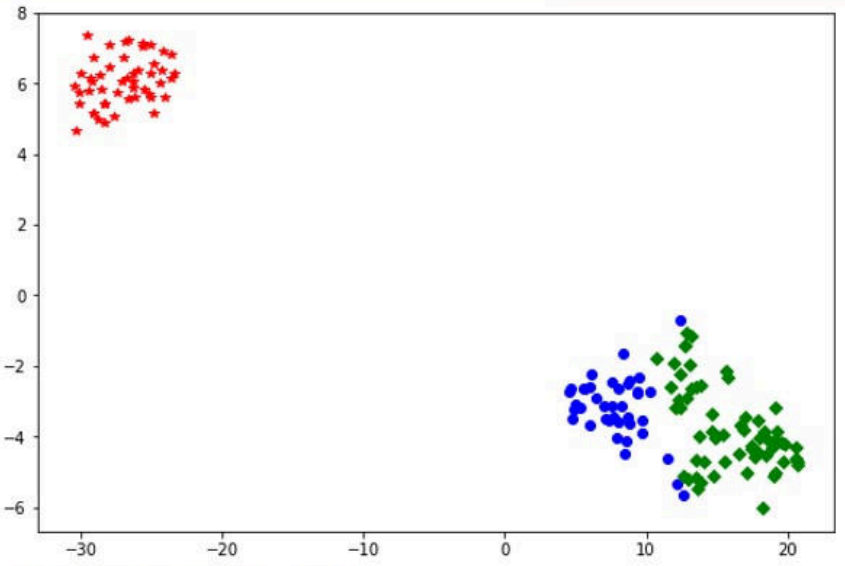
```
import pandas as pd
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
# 使用TSNE进行数据降维，降成两维
tsne = TSNE(n_components=2,init='random', random_state=177).fit(iris_data)
df=pd.DataFrame(tsne.embedding_)        #将原始数据转换为DataFrame
df['labels'] = kmeans.labels_           #将聚类结果存储进df数据表
```

89

使用sklearn估算器构建聚类模型

二维空间查看聚类效果

```
# 提取不同标签的数据
df1 = df[df['labels']==0]
df2 = df[df['labels']==1]
df3 = df[df['labels']==2]
# 绘制图形
fig = plt.figure(figsize=(9,6))    #设定画布
##用不同的颜色表示不同数据
plt.plot(df1[0],df1[1],'bo', df2[0],df2[1],'r*', df3[0],df3[1],'gD')
plt.savefig('tmp/聚类结果.png')
plt.show()      # 显示图片
```



评价聚类模型

聚类模型评价指标

- 聚类评价的标准是组内的对象相互之间是相似的（相关的），而不同组中的对象是不同的（不相关的）。即组内的相似性越大，组间差别越大，聚类效果就越好。sklearn的metrics模块提供了聚类模型评价方法。

方法名称	真实值	最佳值	sklearn函数
ARI评价法（兰德系数）	需要	1.0	adjusted_rand_score
AMI评价法（互信息）	需要	1.0	adjusted_mutual_info_score
V-measure评分	需要	1.0	completeness_score
FMI评价法	需要	1.0	fowlkes_mallows_score
轮廓系数评价法	不需要	畸变程度最大	silhouette_score
Calinski-Harabasz指数评价法	不需要	相较最大	calinski_harabaz_score

评价聚类模型

聚类模型评价指标

- 上表总共列出了6种评价的方法，其中前4种方法均需要真实值的配合才能够评价聚类算法的优劣，后2种则不需要真实值的配合。但是前4种方法评价的效果更具有说服力，并且在实际运行的过程中在有真实值做参考的情况下，聚类方法的评价可以等同于分类算法的评价。
- 除了轮廓系数以外的评价方法，在不考虑业务场景的情况下都是得分越高，其效果越好，最高分值均为1。而轮廓系数则需要判断不同类别数目的情况下其轮廓系数的走势，寻找最优的聚类数目。
- 在具备真实值作为参考的情况下，几种方法均可以很好地评估聚类模型。在没有真实值作为参考的时候，轮廓系数评价方法和Calinski-Harabasz指数评价方法可以结合使用。

评价聚类模型

聚类模型评价指标

- 使用FMI评价法评价K-Means聚类模型

```
from sklearn.metrics import fowlkes_mallows_score     #FMI评价法
for i in range(2,7):          #分别评价K-Means聚类模型聚成2-7类的情况
    kmeans = KMeans(n_clusters = i,random_state=123).fit(iris_data)  #构建并训练模型
    score = fowlkes_mallows_score(iris_target, kmeans.labels_)
    print('iris数据聚%d类FMI评价分值为：%f' %(i,score))
```

聚类之后的标签

鸢尾花数据真实标签

```
iris数据聚2类FMI评价分值为：0.750473
iris数据聚3类FMI评价分值为：0.820808
iris数据聚4类FMI评价分值为：0.753970
iris数据聚5类FMI评价分值为：0.725483
iris数据聚6类FMI评价分值为：0.600691
```

93

评价聚类模型

聚类模型评价指标

- 使用轮廓系数评价法评价K-Means聚类模型

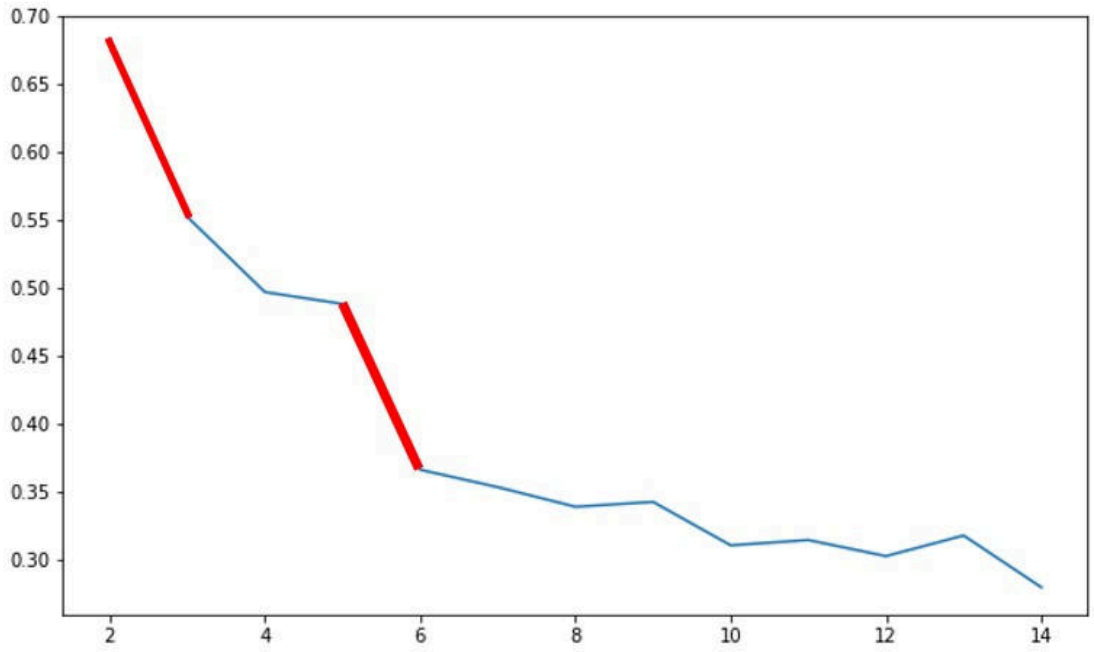
```
from sklearn.metrics import silhouette_score    #轮廓系统评价法
import matplotlib.pyplot as plt
silhouetteScore = []
for i in range(2,15):          #分别评价K-Means聚类模型聚成2-15类的情况
    kmeans = KMeans(n_clusters = i,random_state=123).fit(iris_data)  #构建并训练模型
    score = silhouette_score(iris_data,kmeans.labels_)
    silhouetteScore.append(score)
plt.figure(figsize=(10,6))
plt.plot(range(2,15),silhouetteScore,linewidth=1.5, linestyle="-")
plt.show()
```


雨课堂 Rain Classroom

评价聚类模型

聚类模型评价指标

- 使用轮廓系数评价法评价K-Means聚类模型



95

评价聚类模型

聚类模型评价指标

- 使用Calinski-Harabasz指数评价法评价K-Means聚类模型

```
from sklearn.metrics import calinski_harabaz_score      #Calinski-Harabasz指数评价法
for i in range(2,7):          #分别评价K-Means聚类模型聚成2-7类的情况
    kmeans = KMeans(n_clusters = i,random_state=123).fit(iris_data)  #构建并训练模型
    score = calinski_harabaz_score(iris_data,kmeans.labels_)
    print('iris数据聚%d类calinski_harabaz指数为：%f'%(i,score))
```

```
iris数据聚2类calinski_harabaz指数为： 513.303843
iris数据聚3类calinski_harabaz指数为： 560.399924
iris数据聚4类calinski_harabaz指数为： 529.120719
iris数据聚5类calinski_harabaz指数为： 494.094382
iris数据聚6类calinski_harabaz指数为： 474.753604
```


雨课堂 Rain Classroom

构建并评价聚类模型任务（自我练习）

对Seeds数据构建K-Means聚类模型并评价该模型

1. 对Seeds数据构建K-Means聚类模型

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
seeds = pd.read_csv('data/seeds_dataset.txt', sep = '\t')          # 读取数据集
print('数据集形状为：', seeds.shape)
seeds_data = seeds.iloc[:,:7].values          # 提取数据集中的特征
seeds_target = seeds.iloc[:,7].values         # 提取数据集中的标签
seeds_names = seeds.columns[:7]               # 提取特征名
```

97

构建并评价聚类模型任务（自我练习）

对Seeds数据构建K-Means聚类模型并评价该模型

1. 对Seeds数据构建K-Means聚类模型

```
# 处理数据
stdScale = StandardScaler().fit(seeds_data)                #标准差标准化数据训练模型
seeds_dataScale = stdScale.transform(seeds_data)           #标准差标准化


# 构建并训练模型
kmeans = KMeans(n_clusters=3, random_state=42).fit(seeds_data)
print('构建的KM-eans模型为：\n',kmeans)
```

98

雨课堂 Rain Classroom

构建并评价聚类模型任务（自我练习）

对Seeds数据构建K-Means聚类模型并评价该模型

2. 对构建的K-Means聚类模型进行评价

```
from sklearn.metrics import calinski_harabaz_score      #选取评价方法
for i in range(2,7):  #构建并训练模型
    kmeans = KMeans(n_clusters = i,random_state=12).fit(seeds_data)
    score = calinski_harabaz_score(seeds_data, kmeans.labels_)      #评价模型
    print('seeds数据聚%d类calinski_harabaz指数为：%f'%(i,score))
```

```
seeds数据聚2类calinski_harabaz指数为： 351.179992
seeds数据聚3类calinski_harabaz指数为： 375.804961
seeds数据聚4类calinski_harabaz指数为： 327.835320
seeds数据聚5类calinski_harabaz指数为： 310.331840
seeds数据聚6类calinski_harabaz指数为： 302.473069
```

99

目录


1
使用sklearn转换器处理数据
2
构建并评价聚类模型
3
构建并评价分类模型
4
构建并评价回归模型
5
小结

构建并评价分类模型

任务描述

- 分类是指构造一个分类模型，输入样本的特征值，输出对应的类别，将每个样本映射到预先定义好的类别。分类模型建立在已有类别标记的数据集上，属于有监督学习。
- 在实际应用场景中，分类算法被应用于行为分析、物品识别、图像检测等。

任务分析

（1）使用**sklearn估算器**构建支持向量机（SVM）模型

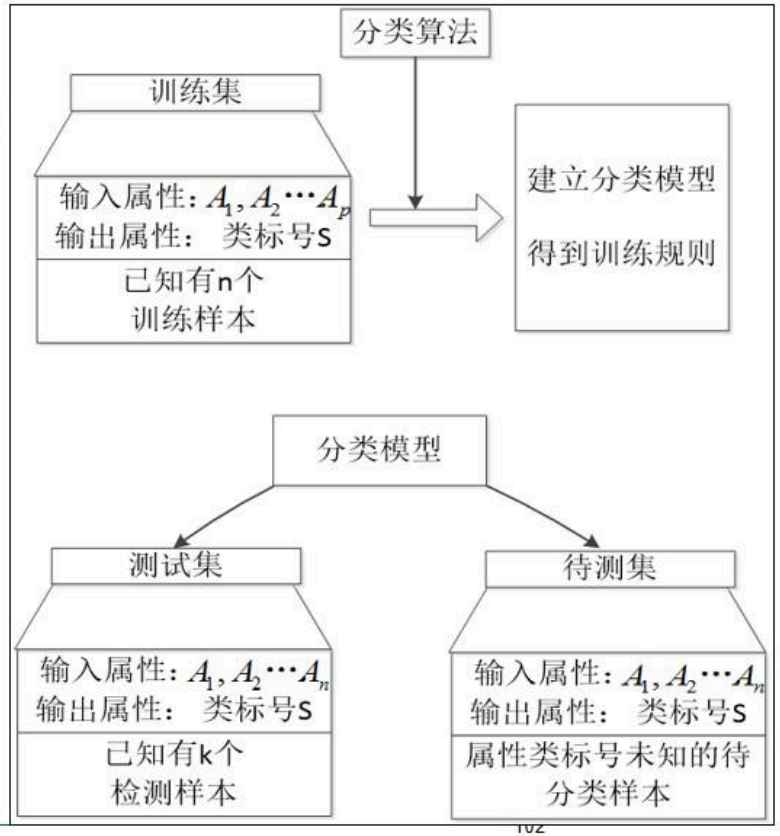
（2）根据**分类模型的评价指标**评价支持向量机（SVM）模型。

101

使用sklearn估算器构建分类模型

分类算法的实现过程

- 在数据分析领域，分类算法有很多，其原理千差万别
  - 有基于样本距离的最近邻算法；
  - 有基于特征信息熵的决策树；
  - 有基于bagging的随机森林；
  - 有基于boosting的梯度提升分类树；
- 但其实现的过程相差不大。过程如图所示。



了解SVM分类算法

1. 基本概念

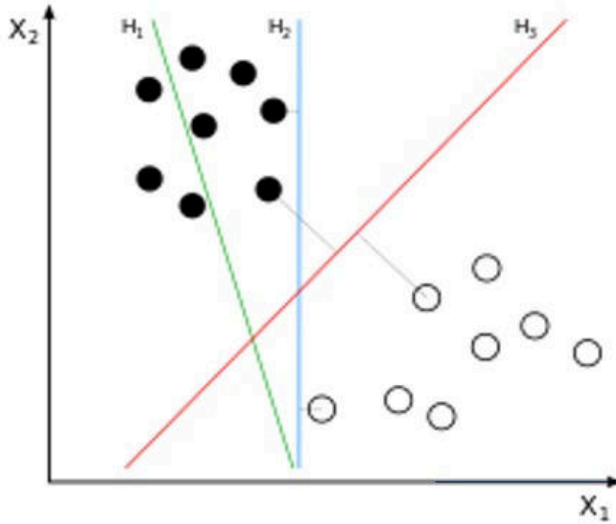
- 支持向量机（support vector machines，SVM）是一种二分类模型；
- 它将实例的特征向量映射为空间中的一些点；
- SVM 的目的就是想要画出一条线，以 “最好地” 区分这两类点，以至如果以后有了新的点，这条线也能做出很好的分类。
- SVM 适合中小型数据样本、非线性、高维的分类问题。

雨课堂 Rain Classroom

了解SVM分类算法

1. 基本概念

- 将实例的特征向量（以二维为例）映射为空间中的一些点，如下图的实心点和空心点，它们属于不同的两类。SVM 的目的就是想要画出一条线，以“最好地”区分这两类点，以至如果以后有了新的点，这条线也能做出很好的分类。



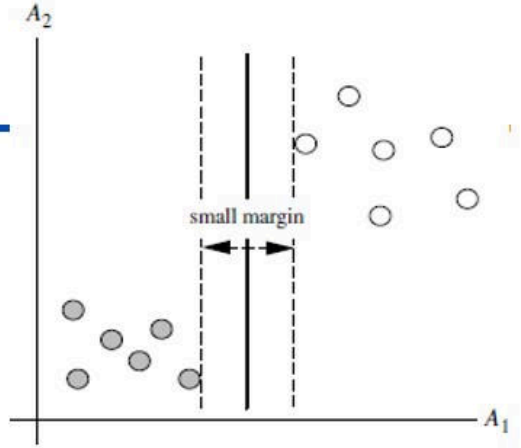
- 线是有无数条可以画的，区别就在于效果好不好，每条线都可以叫做一个划分超平面。
- 比如绿线就不好，蓝线还凑合，红线看起来就比较好。
- 我们所希望找到的这条效果最好的线就是具有“**最大间隔的划分超平面**”。

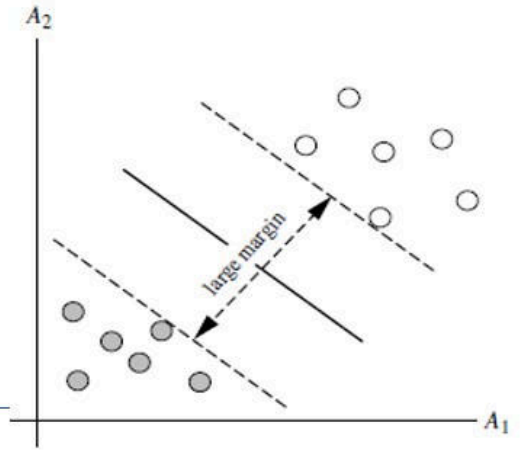
104

了解SVM分类算法

1. 基本概念

- SVM 将会寻找可以区分两个类别并且能使间隔（margin）最大的划分超平面；
- 间隔（margin）是什么？
  - 对于任意一个超平面，其两侧数据点都距离它有一个最小距离（垂直距离），这两个最小距离的和就是间隔。
- 从图可以看出，虚线上的点到划分超平面的距离都是一样的，实际上只有这几个点共同确定了超平面的位置，因此被称作 “支持向量（support vectors）”，“支持向量机” 也是由此来的。





使用sklearn估计器构建分类模型

加载所需的函数

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
cancer = load_breast_cancer()
cancer_data = cancer['data']
cancer_target = cancer['target']
cancer_names = cancer['feature_names']
```

106

雨课堂 Rain Classroom

使用sklearn估计器构建分类模型

数据预处理

```
# 将数据划分为训练集测试集
cancer_data_train, cancer_data_test, cancer_target_train, cancer_target_test = \
    train_test_split(cancer_data,cancer_target, test_size = 0.2,random_state = 22)

# 数据标准化（标准差标准化）
stdScaler = StandardScaler().fit(cancer_data_train)   #使用sklearn的转换器进行数据预处理(标准化)
cancer_trainStd = stdScaler.transform(cancer_data_train)
cancer_testStd = stdScaler.transform(cancer_data_test)
```


雨课堂 Rain Classroom

使用sklearn估计器构建分类模型

构建SVM模型

```
svm = SVC().fit(cancer_trainStd, cancer_target_train)        # 有监督学习,使用估计器训练模型,要传入标签
print('建立的SVM模型为：\n',svm)
# 预测测试集结果
cancer_target_pred = svm.predict(cancer_testStd)
print('预测前20个结果为：\n', cancer_target_pred[:20])
```

```
建立的SVM模型为：
 SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,
  decision_function_shape='ovr', degree=3, gamma='auto', kernel='rbf',
  max_iter=-1, probability=False, random_state=None, shrinking=True,
  tol=0.001, verbose=False)
预测前20个结果为：
 [1 0 0 0 1 1 1 1 1 1 1 1 0 1 1 1 0 0 1 1]
```


雨课堂 Rain Classroom

使用sklearn估计器构建分类模型

预测准确性评估

- 将预测结果和真实结果做比对，求出预测对的结果和预测错的结果，并求出准确率。

```
# 求出预测和真实一样的数目
true = np.sum(cancer_target_pred == cancer_target_test )
print('预测对的结果数目为：', true)
print('预测错的的结果数目为：', cancer_target_test.shape[0]-true)
print('预测结果准确率为：', true/cancer_target_test.shape[0])
```

```
预测对的结果数目为： 111
预测错的的结果数目为： 3
预测结果准确率为： 0.9736842105263158
```


雨课堂 Rain Classroom

使用sklearn估计器构建分类模型

sklearn库常用分类算法函数

- sklearn中提供的分类算法非常多，分别存在于不同的模块中。常用的分类算法如下表所示。

模块名称	函数名称	算法名称
linear_model	LogisticRegression	Logistic回归
svm	SVC	支持向量机
neighbors	KNeighborsClassifier	K最近邻分类
naive_bayes	GaussianNB	高斯朴素贝叶斯
tree	DecisionTreeClassifier	分类决策树
ensemble	RandomForestClassifier	随机森林分类
ensemble	GradientBoostingClassifier	梯度提升分类树

雨课堂 Rain Classroom

评价分类模型

分类模型的评价指标

- 分类模型对测试集进行预测而得出的准确率并不能很好地反映模型的性能，为了有效判断一个预测模型的性能表现，需要结合真实值，计算出精确率、召回率、F1值和Cohen's Kappa系数等指标来衡量。
- 常规分类模型的评价指标如表所示。分类模型评价方法前4种都是分值越高越好，其使用方法基本相同。

方法名称	最佳值	sklearn函数
Precision（精确率）	1.0	metrics.precision_score
Recall（召回率）	1.0	metrics.recall_score
F1值	1.0	metrics.f1_score
Cohen's Kappa系数	1.0	metrics.cohen_kappa_score
ROC曲线	最靠近y轴	metrics. roc_curve

雨课堂 Rain Classroom

评价分类模型

分类模型的评价指标

- Precision（精确率）、Recall（召回率）、F1值、AUC和ROC曲线其实都是评价模型好坏的指标，而且相互之间是有关系的，只是侧重点不同，下面就用一个例子解释这些指标。
- 以一个有监督的二分类预测模型为例，模型对每个样本的预测结果为一个概率值，我们从中选取一个阈值来区分为预测结果为正值，还是负值。这样就可以得出混淆矩阵(Confusion matrix)

混淆矩阵		预测表现	
		0	1
实际表现	0	True Negative 真阴性（TN）	False Positive 假阳性（FP）
	1	False Negative 假阴性（FN）	True Positive 真阳性（TP）

F1 = 2PR/(P+R)

F1相当于精确率和召回率的综合评价指标

召回率 R = TP/(TP+FN)

精确率 P = TP/(TP+FP)

雨课堂 Rain Classroom

评价分类模型

分类模型的评价指标

- ROC曲线的横坐标是伪阳性率，纵坐标是真阳性率。
- 一个二分类模型的阈值可能设定为高或低，每种阈值的设定会得出不同的 FPR 和 TPR ，将同一模型每个阈值的 (FPR, TPR) 坐标都画在 ROC 空间里，就成为特定模型的ROC曲线。

混淆矩阵		预测表现	
		0	1
实际表现	0	True Negative 真阴性（TN）	False Positive 假阳性（FP）
	1	False Negative 假阴性（FN）	True Positive 真阳性（TP）

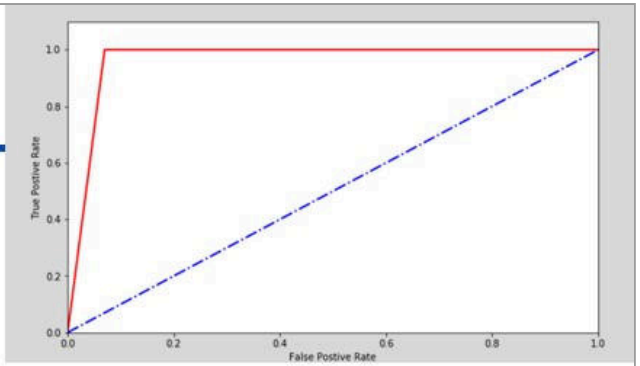
伪阳性率 FPR = FP/(TN+FP)

真阳性率 TPR = TP/(TP+FN)

雨课堂 Rain Classroom

评价分类模型

分类模型的评价指标



- ROC曲线的意义

1. ROC曲线能很容易的查出任意阈值对学习器的泛化性能影响。
2. 有助于选择最佳的阈值
   - ROC曲线越靠近左上角，模型的准确性就越高。最靠近左上角的ROC曲线上的点是分类错误最少的最好阈值，其假正例和假反例总数最少。
3. 可以对不同的学习器比较性能
   - 将各个学习器的ROC曲线绘制到同一坐标中，直观地鉴别优劣，靠近左上角的ROC曲所代表的学习器准确性最高。

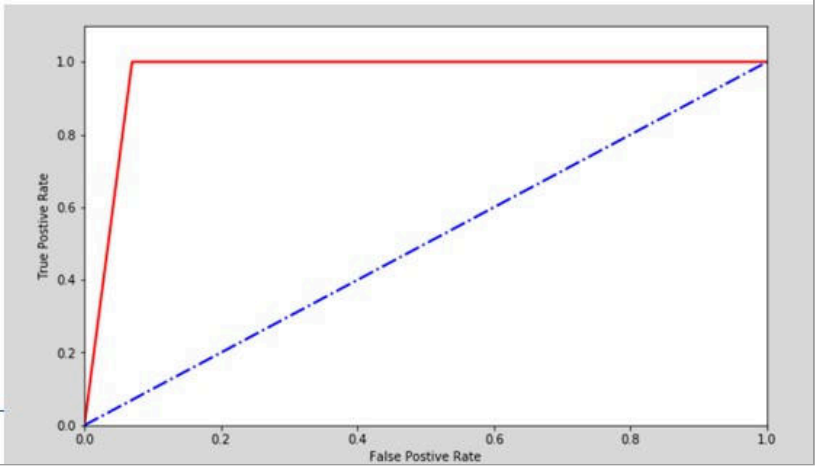
雨课堂 Rain Classroom

评价分类模型

分类模型的评价指标

- AUC曲线

1. AUC（Area Under Curve）被定义为ROC曲线下的面积。
2. 使用AUC值作为模型的评价标准是因为很多时候ROC曲线并不能清晰的说明哪个分类器的效果更好(曲线有高有低，可能有交叉)
3. 作为一个数值，对应AUC更大的分类器效果更好。



评价分类模型

分类模型的评价指标

- 从AUC 判断分类器（预测模型）优劣的标准：
  1. AUC = 1，是完美分类器。
  2. AUC = [0.85, 0.95]，效果很好
  3. AUC = [0.7, 0.85]，效果一般
  4. AUC = [0.5, 0.7]，效果较低，但用于预测股票已经很不错了
  5. AUC = 0.5，跟随机猜测一样（例：丢铜板），模型没有预测价值。
  6. AUC < 0.5，比随机猜测还差；但只要总是反预测而行，就优于随机猜测。

雨课堂 Rain Classroom

评价分类模型

分类模型的评价实例

```
from sklearn.metrics import accuracy_score,precision_score,
                            recall_score,f1_score,cohen_kappa_score
print('使用SVM预测breast_cancer数据的准确率为：',
      accuracy_score(cancer_target_test,cancer_target_pred))
print('使用SVM预测breast_cancer数据的精确率为：',
      precision_score(cancer_target_test,cancer_target_pred))
print('使用SVM预测breast_cancer数据的召回率为：',
      recall_score(cancer_target_test,cancer_target_pred))
print('使用SVM预测breast_cancer数据的F1值为：',
      f1_score(cancer_target_test,cancer_target_pred))
print('使用SVM预测breast_cancer数据的Cohen’s Kappa系数为：',
      cohen_kappa_score(cancer_target_test,cancer_target_pred))
```

117

评价分类模型

分类模型的评价实例

```
from sklearn.metrics import accuracy_score,precision_score,
                            recall_score,f1_score,cohen_kappa_score
print('使用SVM预测breast_cancer数据的准确率为：',
      accuracy_score(cancer_target_test,cancer_target_pred))
print('使用SVM预测breast_cancer数据的精确率为：',
      precision_score(cancer_target_test,cancer_target_pred))
print('使用SVM预测breast_cancer数据的召回率为：',
      recall_score(cancer_target_test,cancer_target_pred))
```

```
使用SVM预测breast_cancer数据的准确率为： 0.9736842105263158
使用SVM预测breast_cancer数据的精确率为： 0.9594594594594594
使用SVM预测breast_cancer数据的召回率为： 1.0
使用SVM预测breast_cancer数据的F1值为： 0.9793103448275862
使用SVM预测breast_cancer数据的Cohen's Kappa系数为： 0.9432082364662903
```

118

评价分类模型

分类模型的评价指标

- sklearn的metrics模块除了提供准确率、精确率、召回率等单一评价指标的函数外，还提供了一个能够输出分类模型评价报告的函数classfication_report，代码如下：

```
from sklearn.metrics import classification_report
print('使用SVM预测iris数据的分类报告为：\n',
      classification_report(cancer_target_test, cancer_target_pred))
```

```
使用SVM预测breast_cancer数据的分类报告为：
              precision    recall  f1-score   support

           0       1.00      0.93      0.96        43
           1       0.96      1.00      0.98        71

avg / total       0.97      0.97      0.97       114
```


雨课堂 Rain Classroom

评价分类模型

ROC曲线

- 除了使用数值，表格形式评估分类模型的性能，还可通过绘制ROC曲线的方式来评估分类模型。

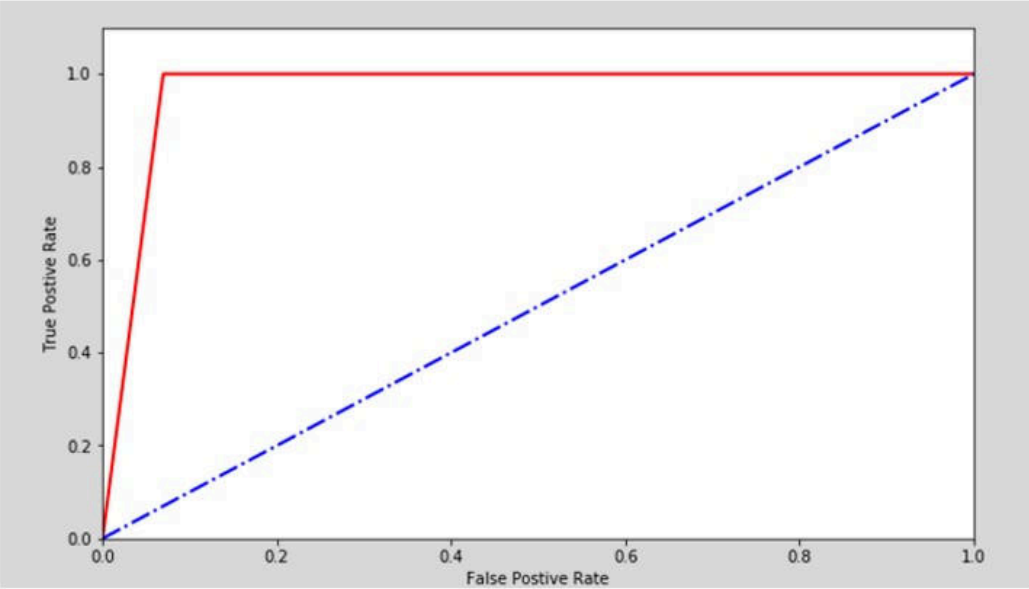
```
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt
# 求出ROC曲线的x轴和y轴
fpr, tpr, thresholds = roc_curve(cancer_target_test, cancer_target_pred)
plt.figure(figsize=(10,6))
plt.xlim(0,1)                                    # 设定x轴的范围
plt.ylim(0.0,1.1)                                # 设定y轴的范围
plt.xlabel('False Postive Rate')                 # 伪阳性率
plt.ylabel('True Postive Rate')                  # 真阳性率
plt.plot(fpr,tpr,linewidth=2, linestyle="-",color='red')
plt.show()
```


雨课堂 Rain Classroom

评价分类模型

ROC曲线

- ROC曲线横纵坐标范围为[0,1]，通常情况下ROC曲线与X轴形成的面积越大（AUC面积值），表示模型性能越好。但是当ROC曲线处于下图中蓝色虚线的位置，就表明了模型的计算结果基本都是随机得来的，在此种情况下模型起到的作用几乎为零。故在实际中ROC曲线离图中蓝色虚线越远表示模型效果越好。



构建并评价分类模型（自我练习）

任务实现

- 常规的鲍鱼的年龄是通过显微镜查看切削、染色后的外壳上的环的数量得到的，十分耗时。
- 一些常规的物理量（如性别、长度、高度、壳体重量等）的测量十分容易获得，若能够使用这些物理量预测年龄将节省大量时间，使用sklearn构建鲍鱼预测的步骤如下。

1. 构建SVM分类模型
2. 评价构建的SVM分类模型

雨课堂 Rain Classroom

构建并评价分类模型（自我练习）

任务实现

1. 加载所需的函数

```
import pandas as pd
from sklearn.svm import SVC
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
abalone = pd.read_csv('data/abalone.data', sep=',')
# 将数据和标签拆开
abalone_data = abalone.iloc[:,:8]          #提取鲍鱼数据中的特征数据
abalone_target = abalone.iloc[:,8]         #提取鲍鱼数据中的特征标签
```

123

构建并评价分类模型（自我练习）

任务实现

2. 数据预处理

```
# 连续型特征离散化
sex = pd.get_dummies(abalone_data['sex'])
abalone_data = pd.concat([abalone_data,sex], axis = 1 )
abalone_data.drop('sex', axis = 1, inplace = True)
# 划分训练集，测试集
abalone_train,abalone_test, abalone_target_train,abalone_target_test = \
train_test_split(abalone_data,abalone_target, train_size=0.8, random_state = 42)
# 标准化
stdScaler = StandardScaler().fit(abalone_train)
abalone_std_train = stdScaler.transform(abalone_train)
abalone_std_test = stdScaler.transform(abalone_test)
```

124

雨课堂 Rain Classroom

构建并评价分类模型（自我练习）

任务实现

3. 构建SVM分类模型

```
svm_abalone = SVC().fit(abalone_std_train, abalone_target_train)
print('建立的SVM模型为：\n', svm_abalone)
```

```
建立的SVM模型为：
 SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,
  decision_function_shape='ovr', degree=3, gamma='auto', kernel='rbf',
  max_iter=-1, probability=False, random_state=None, shrinking=True,
  tol=0.001, verbose=False)
```

125

构建并评价分类模型（自我练习）

任务实现

4. 评价构建的SVM分类模型

```
abalone_target_pred = svm_abalone.predict(abalone_std_test)
print('abalone数据集的SVM分类报告为：\n',
classification_report(abalone_target_test,abalone_target_pred))
```

```
...:       classification_report(abalone_target_test,abalone_target_pred))
abalone数据集的SVM分类报告为：
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         3
           4       0.45      0.69      0.55        13
           5       0.54      0.22      0.31        32
           6       0.40      0.33      0.36        48
           7       0.41      0.44      0.42        84
           8       0.37      0.36      0.37        99
           9       0.28      0.57      0.38       142
          10       0.24      0.33      0.28       139
          11       0.25      0.25      0.25        93
          12       0.00      0.00      0.00        51
```

126

目录


1
使用sklearn转换器处理数据
2
构建并评价聚类模型
3
构建并评价分类模型
4
构建并评价回归模型
5
小结

127

构建并评价回归模型

任务描述

- 回归算法的实现过程与分类算法类似，原理相差不大。
- 分类和回归的主要区别在于，分类算法的标签是离散的，但是回归算法的标签是连续的。
- 回归算法在交通、物流、社交网络和金融领域都能发挥巨大作用。

任务分析

（1）使用**sklearn估算器**构建线性回归（Linear Regression）模型

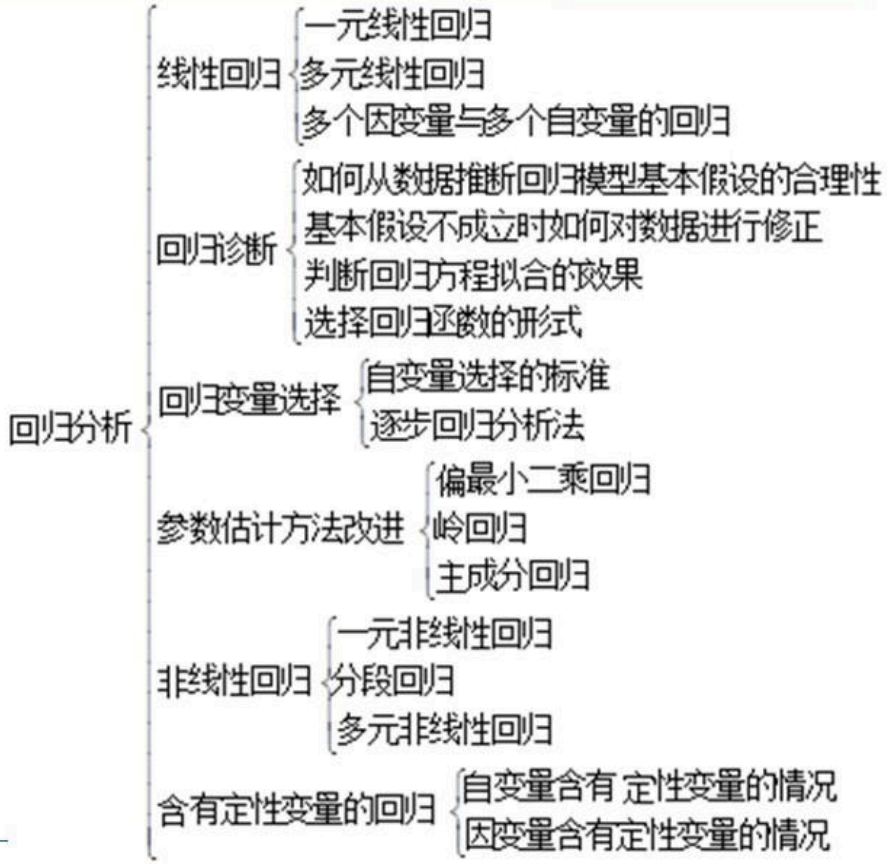
（2）根据**回归模型评价指标**评价线性回归模型

雨课堂 Rain Classroom

使用sklearn估算器构建回归模型

回归分析方法

- 从19世纪初高斯提出最小二乘估计法算起，回归分析的历史已有200多年。
- 从经典的回归分析方法到近代的回归分析方法，按照研究方法划分，回归分析研究的范围大致如图所示。
- 回归算法的实现步骤和分类算法基本相同，分为**学习和预测**2个步骤：
  - 学习是通过训练样本数据来拟合回归方程；
  - 预测则是利用学习过程中拟合出的回归方程，将测试数据放入方程中求出预测值。



使用sklearn估算器构建回归模型

常用的回归模型

回归模型名称	适用条件	算法描述
线性回归	因变量与自变量是线性关系	对一个或多个自变量和因变量之间的线性关系进行建模，可用最小二乘法求解模型系数。
非线性回归	因变量与自变量之间不都是线性关系	对一个或多个自变量和因变量之间的非线性关系进行建模。如果非线性关系可以通过简单的函数变换转化成线性关系，用线性回归的思想求解；如果不能转化，用非线性最小二乘方法求解。
Logistic回归	因变量一般有1和0（是与否）两种取值	是广义线性回归模型的特例，利用Logistic函数将因变量的取值范围控制在0和1之间，表示取值为1的概率。
岭回归	参与建模的自变量之间具有多重共线性	是一种改进最小二乘估计的方法。
主成分回归	参与建模的自变量之间具有多重共线性	主成分回归是根据主成分分析的思想提出来的，是对最小二乘法的一种改进，它是参数估计的一种有偏估计。可以消除自变量之间的多重共线性。

使用sklearn估算器构建回归模型

sklearn库常用回归算法函数

- sklearn内部提供了不少回归算法，常用的函数如下表所示。

模块名称	函数名称	算法名称
linear_model	LinearRegression	线性回归
svm	SVR	支持向量回归
neighbors	KNeighborsRegressor	最近邻回归
tree	DecisionTreeRegressor	回归决策树
ensemble	RandomForestRegressor	随机森林回归
ensemble	GradientBoostingRegressor	梯度提升回归树

雨课堂 Rain Classroom

使用sklearn估算器构建boston数据集线性回归模型

加载所需函数

```
from sklearn.linear_model import LinearRegression
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
# 加载boston数据
boston = load_boston()
X = boston['data']                    # 提取boston数据集中的特征数据
y = boston['target']                  # 提取boston数据集中的特征标签
boston_names = boston['feature_names']  # 提取boston数据集中的特征名称
# 将数据划分为训练集、测试集
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.2,random_state=125)
```


雨课堂 Rain Classroom

使用sklearn估算器构建boston数据集线性回归模型

建立线性回归模型（监督学习）

```
clf = LinearRegression().fit(X_train, y_train)
print('建立的LinearRegression模型为：\n', clf)
y_pred = clf.predict(X_test)          # 预测训练集结果
print('预测前20个结果为：\n', ,y_pred[:20])
```

```
建立的LinearRegression模型为：
 LinearRegression(copy_X=True, fit_intercept=True, n_jobs=1, normalize=False)
预测前20个结果为：
 [21.12953164 19.67578799 22.01735047 24.62046819 14.45164813 23.32325459
 16.6468677  14.9175848  33.58466804 17.48328609 25.50385719 36.60215179
 25.95309333 28.48503161 19.34928078 20.16966217 25.9788081  18.25959831
 16.52754056 17.08448854]
```


雨课堂 Rain Classroom

使用sklearn估算器构建boston数据集线性回归模型

模型回归结果可视化

- 可以利用预测结果和真实结果画出折线图作对比，以便更直观看出线性回归模型效果。

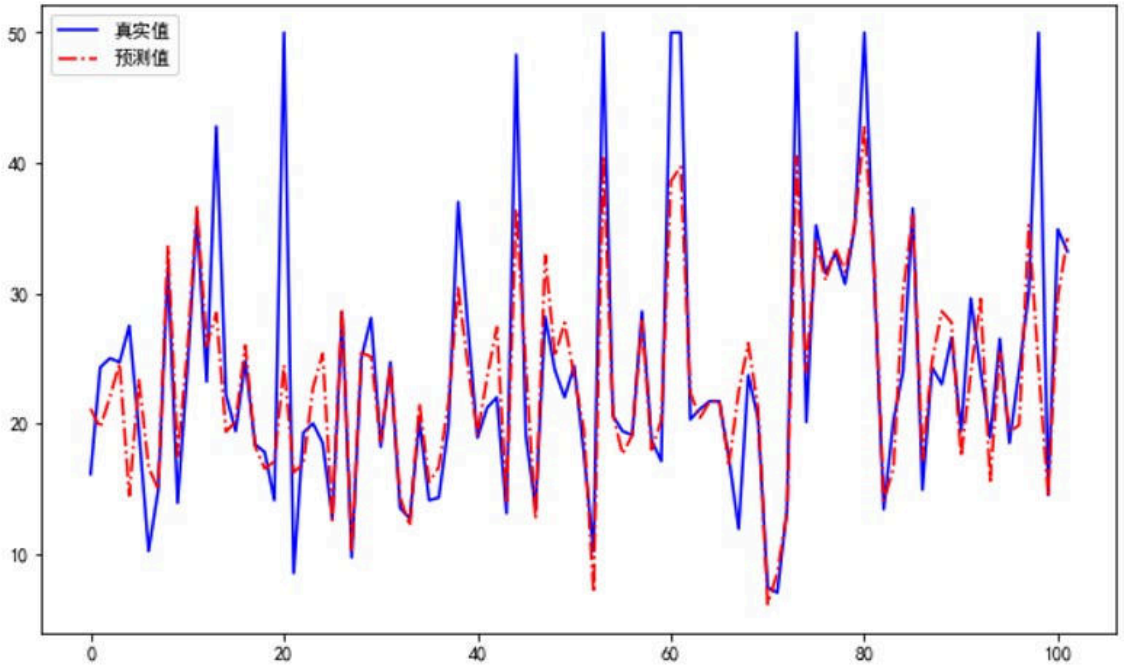
```
import matplotlib.pyplot as plt
from matplotlib import rcParams
rcParams['font.sans-serif'] = 'SimHei'
fig = plt.figure(figsize=(10,6))     # 设定空白画布，并制定大小
# 用不同的颜色表示不同数据
plt.plot(range(y_test.shape[0]),y_test,color="blue", linewidth=1.5, linestyle="-")
plt.plot(range(y_test.shape[0]),y_pred,color="red", linewidth=1.5, linestyle="-.")
plt.legend(['真实值','预测值'])
plt.savefig('../tmp/聚类结果.png')
plt.show()                  # 显示图片
```


雨课堂 Rain Classroom

使用sklearn估算器构建boston数据集线性回归模型

模型回归结果可视化

- 可以利用预测结果和真实结果画出折线图作对比，以便更直观看出线性回归模型效果。



评价回归模型

回归模型评价指标

- 回归模型的性能评估不同于分类模型，虽然都是对照真实值进行评估，但由于回归模型的预测结果和真实值都是连续的，所以不能够求取Precision、Recall和F1值等评价指标。**回归模型拥有一套独立的评价指标。**
- 常用的回归模型评价指标如下表所示。平均绝对误差、均方误差和中值绝对误差的值越靠近0，模型性能越好。可解释方差值和R方值则越靠近1，模型性能越好。

方法名称	最优值	sklearn函数
平均绝对误差	0.0	metrics. mean_absolute_error
均方误差	0.0	metrics. mean_squared_error
中值绝对误差	0.0	metrics. median_absolute_error
可解释方差值	1.0	metrics. explained_variance_score
R方值	1.0	metrics. r2_score

评价回归模型

实例

```
# 导入平均绝对误差，均方误差，中值绝对误差，R²值，可解释方差值评估模块
from sklearn.metrics import mean_absolute_error, mean_squared_error, \
                            median_absolute_error,explained_variance_score, r2_score
print('Boston数据线性回归模型的平均绝对误差为：', mean_absolute_error(y_test, y_pred))
print('Boston数据线性回归模型的均方误差为：', mean_squared_error(y_test, y_pred))
print('Boston数据线性回归模型的中值绝对误差为：', median_absolute_error(y_test, y_pred))
print('Boston数据线性回归模型的可解释方差值为：', explained_variance_score(y_test,
y_pred))
print('Boston数据线性回归模型的R方值为：', r2_score(y_test, y_pred))
```


雨课堂 Rain Classroom

评价回归模型

实例

```
# 导入平均绝对误差，均方误差，中值绝对误差，R²值，可解释方差值评估模块
from sklearn.metrics import mean_absolute_error, mean_squared_error, \
                            median_absolute_error,explained_variance_score, r2_score
print('Boston数据线性回归模型的平均绝对误差为：', mean_absolute_error(y_test, y_pred))
print('Boston数据线性回归模型的均方误差为：', mean_squared_error(y_test, y_pred))
print('Boston数据线性回归模型的中值绝对误差为：', median_absolute_error(y_test, y_pred))
                                                                re(y_test, y_pred))
```

```
Boston数据线性回归模型的平均绝对误差为：  3.3776426973628 18
Boston数据线性回归模型的均方误差为：  31.15059667690504
Boston数据线性回归模型的中值绝对误差为：  1.7774213157360652
Boston数据线性回归模型的可解释方差值为：  0.7105949626282904
Boston数据线性回归模型的R方值为：  0.7068954225782409
```

138

构建并评价回归模型任务（自我练习）

1. 构建加利福尼亚住房数据回归模型

```
import pandas as pd
from sklearn.ensemble import GradientBoostingRegressor     # 梯度提升回归模型
from sklearn.model_selection import train_test_split
house = pd.read_csv('data/cal_housing.data',sep=',')
house_data = house.iloc[:, :-1]
house_target = house.iloc[:, -1]
house_names = ['longitude','latitude','housingMedianAge', 'totalRooms',
                'totalBedrooms','population', 'households', 'medianIncome']
house_train,house_test,house_target_train,house_target_test = \
    train_test_split(house_data,house_target,test_size = 0.2, random_state = 42)
GBR_house = GradientBoostingRegressor().fit(house_train,house_target_train)
print('建立的梯度提升回归模型为：\n', GBR_house)
```

139

雨课堂 Rain Classroom

构建并评价回归模型任务（自我练习）

2. 评价构建的加利福尼亚住房数据回归模型

```
house_target_pred = GBR_house.predict(house_test)

#对回归模型进行评估，导入平均绝对误差，均方误差，中值绝对误差，R2值，可解释方差值评估模块
from sklearn.metrics import mean_absolute_error,mean_squared_error, \
                            median_absolute_error, explained_variance_score,r2_score
print('回归树模型的平均绝对误差：', mean_absolute_error(house_target_test,house_target_pred))
print('回归树模型的均方误差：', mean_squared_error(house_target_test,house_target_pred))
print('回归树模型的中值绝对误差为：', median_absolute_error(house_target_test,house_target_pred))
print('回归树模型的可解释方差值为：',explained_variance_score(house_target_test, house_target_pred))
print('california_housing数据梯度提升回归树模型的R方值为：',
                            r2_score(house_target_test,house_target_pred))
```

140

雨课堂 Rain Classroom

构建并评价回归模型任务（自我练习）

2. 评价构建的加利福尼亚住房数据回归模型

```
house_target_pred = GBR_house.predict(house_test)

#对回归模型进行评估，导入平均绝对误差，均方误差，中值绝对误差，R2值，可解释方差值评估模块
from sklearn.metrics import mean_absolute_error,mean_squared_error,median_absolute_error, \
                            explained_variance_score,r2_score
print('回归树模型的平均绝对误差为：', mean_absolute_error(house_target_test,house_target_pred))
print('回归树模型的均方误差为：', mean_squared_error(house_target_test,house_target_pred))
```

```
california_housing数据梯度提升回归树模型的平均绝对误差为：  38056.75815557929
california_housing数据梯度提升回归树模型的均方误差为：  3103233005.5273113
california_housing数据梯度提升回归树模型的中值绝对误差为：  26179.478445698274
california_housing数据梯度提升回归树模型的可解释方差值为：  0.7618952080246163
california_housing数据梯度提升回归树模型的R方值为：  0.7618473592138286
```

构建并评价回归模型任务（自我练习）

2. 评价构建的加利福尼亚住房数据回归模型

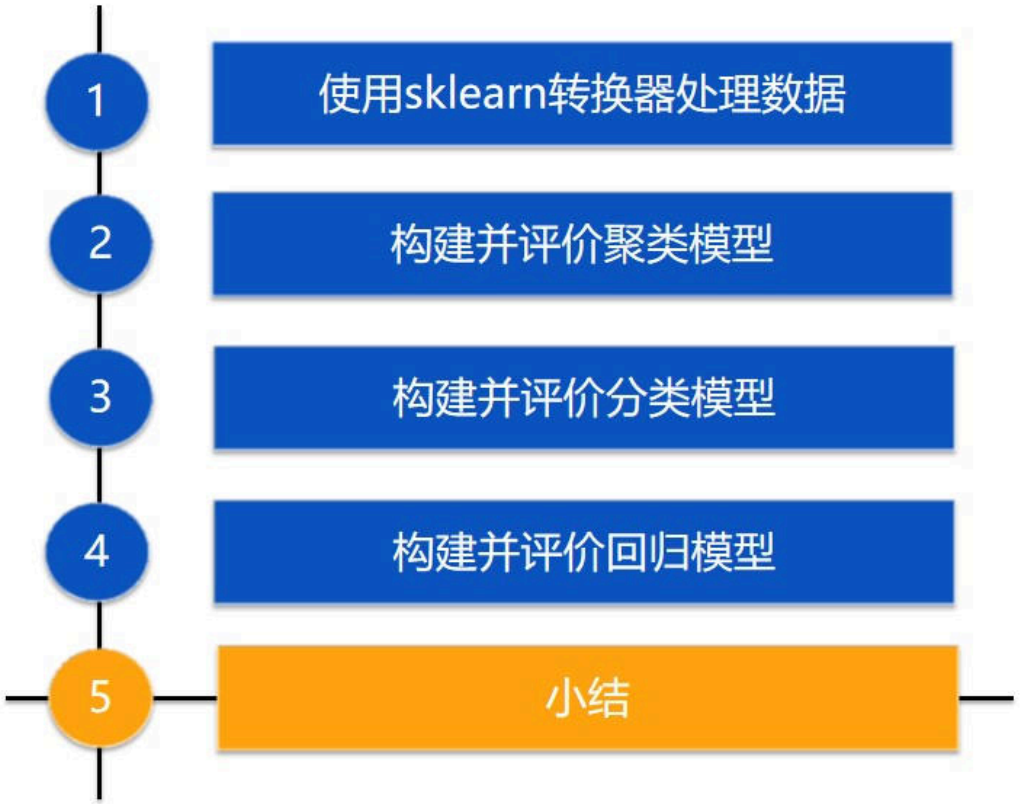
```
#尝试使用随机森林回归
from sklearn.ensemble import RandomForestRegressor
RFR_house = RandomForestRegressor().fit(house_train,house_target_train)
print('建立的随机森林回归模型为：\n', RFR_house)
house_target_pred = RFR_house.predict(house_test)
```

```
california_housing数据随机森林回归模型的平均绝对误差为：  34279.96727228681
california_housing数据随机森林回归模型的均方误差为：  2815471138.9420857
california_housing数据随机森林回归模型的中值绝对误差为：  20175.0
california_housing数据随机森林回归模型的可解释方差值为：  0.7841673132372502
california_housing数据随机森林回归模型的R方值为：  0.7839311822212423
```

- 尝试用随机森林构建回归模型，并评价，从评价指标对比发现它比梯度提升回归模型更好。

雨课堂 Rain Classroom

目录


1
使用sklearn转换器处理数据
2
构建并评价聚类模型
3
构建并评价分类模型
4
构建并评价回归模型
5
小结

小结

本章主要根据数据分析的应用分类，重点介绍了对应的数据分析建模方法及实现过程。

- sklearn数据分析技术的基本任务主要体现在聚类、分类和回归三类。
- 每一类又有对应的多种评估方法，能够评价所构建模型的性能优劣。

通过这一章的学习，基本能够掌握常用的模型构建与评估方法，可在以后的数据分析过程中采用适当的算法并按所介绍的步骤实现综合应用。

雨课堂 Rain Classroom